

ภาพรวมของบริการ (Service Overview)

MissionProgress Service

1. Service Owner

- นายรัฐธรรมนุญ โคสาแสง (Ratthatummanoon Kosasang)
- รหัสนักศึกษา 6609612178

2. Service Purpose

MissionProgress Service คือบริการสำหรับทีมกู้ภัย (Rescue Team) เพื่อใช้ในการรายงานความคืบหน้าของภารกิจที่ได้รับมอบหมาย อัปเดตสถานะของเหตุการณ์ (Incident Status) และบันทึกรายละเอียดการปฏิบัติงานหน้างาน (Action Logs) เพื่อให้ศูนย์สั่งการได้รับข้อมูลที่ถูกต้องและเป็นปัจจุบันที่สุด

3. Pain Point ที่แก้ไข

ปัจจุบันศูนย์สั่งการ (Command Center) ขาดการมองเห็นภาพรวมและการติดตามสถานะการทำงานของทีมกู้ภัยแบบเรียลไทม์ (Lack of real-time visibility) รวมถึงขาดข้อมูลประเมินความรุนแรง (Impact Assessment) ที่ถูกต้องแม่นยำจากหน้างานจริง ทำให้การตัดสินใจสั่งการหรือสนับสนุนทรัพยากรเกิดความล่าช้าและผิดพลาด

4. Target Users

- Rescue Team (ผู้ใช้งานหลัก): ใช้สำหรับอัปเดตสถานะ แจ้งพิกัดเมื่อถึงหน้างาน และประเมินความรุนแรง
- Dispatcher: ใช้ดูข้อมูล Timeline การทำงานเพื่อติดตามผล (ผ่านการดึงข้อมูลไปแสดงผล)

5. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
 - State Transition Management
 - รับผิดชอบการเปลี่ยนสถานะภารกิจตาม Workflow: DISPATCHED → EN_ROUTE → ON_SITE → NEED_BACKUP → RESOLVED พร้อม Validate ว่าการเปลี่ยนสถานะสมเหตุสมผล
 - Timeline / Action Log Recording
 - บันทึก Log การปฏิบัติงานทุกรายการ เช่น เวลาที่ถึงจุดเกิดเหตุ, การกระทำ (Evacuation start, First aid applied), ผู้กระทำ
 - Field Assessment Forwarding
 - รับข้อมูลประเมิน Impact Level / Priority จากทีมกู้ภัยหน้างาน บันทึกเป็น Action Log แล้ว Publish Event ไปยัง IncidentTracking (ผู้เป็นเจ้าของข้อมูล) เพื่ออัปเดตค่าจริง
 - Evidence Image Management
 - รับและจัดเก็บหลักฐานภาพถ่ายจากหน้างาน (Evidence Images) ผ่าน S3 Presigned URL พร้อมเชื่อม Image Key กับ Timeline
 - Event Publishing
 - Publish Domain Events (MissionStatusChangedEvent, FieldAssessmentUpdatedEvent) ไปยัง SNS เพื่อแจ้ง Service อื่นๆ
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
 - การ "สั่งการ" หรือ "มอบหมายงาน" ให้ทีมกู้ภัย
 - Manage Dispatch Service
 - การค้นหาเส้นทาง
 - SafeRoute Service
 - การจัดการทรัพยากรโรงพยาบาล / การส่งตัวผู้ป่วย
 - HospitalResourceStatus Service

- การเป็น Source of Truth ของ Impact Level / Priority / Location
 - IncidentTracking Service (MissionProgress แยก Forward ข้อมูล)

6. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

1. Status Validation (การตรวจสอบสถานะ):

- ตรวจสอบว่าการเปลี่ยนสถานะสมเหตุสมผลหรือไม่ตามตาราง State Transition ที่กำหนด
- เช่น ต้องเป็น ON_SITE ก่อนจึงจะ RESOLVED ได้
- NEED_BACKUP จะ Trigger การ Publish Event เพื่อแจ้งเตือน

2. Field Assessment Acceptance (การรับข้อมูลจากหน่วยงาน):

- รับข้อมูล Impact Level / Priority ที่ทีมกู้ภัยปรับจากหน่วยงาน ได้ทันที โดยไม่ต้องรอการอนุมัติ
- บันทึกลง Timeline เป็น Action Log พร้อม Forward ผ่าน Event ให้ IncidentTracking อัปเดต

3. Idempotency Decision (การตัดสินใจเรื่อง Duplicate):

- ตรวจสอบ Idempotency Key ว่า Request นี้เคยถูกประมวลผลแล้วหรือยัง
- ถ้าเคย → Return cached response ทันทีโดยไม่ประมวลผลซ้ำ

การตัดสินใจอิงจาก:

- Input จากทีมกู้ภัยหน่วยงาน (User Action)
- สถานะปัจจุบันของภารกิจ (Current State ใน DynamoDB)
- Idempotency Records (ใน DynamoDB)

บริการตัดสินใจได้เองภายใต้ Business Rules ที่กำหนด โดยไม่ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

7. Owned Data

ข้อมูลที่บริการนี้เป็นเจ้าของและดูแลโดยตรง (Source of Truth)

- Mission Timeline / Action Logs: ข้อมูลประวัติการทำงานทั้งหมดที่เป็น Array of objects (เก็บ เวลา, เหตุการณ์, รายละเอียด, ผู้กระทำ)
- Operational Context Updates: ข้อมูลบริบทหน่วยงานล่าสุดที่ทีมกู้ภัยส่งมา เช่น last_update_at, updated_by (Rescue Unit ID)

8. Linked Data (Reference Only)

ข้อมูลที่บริการนี้ต้องอ้างอิงจากบริการอื่น แต่ไม่ได้เป็นเจ้าของหลัก

- Incident Master Data: อ้างอิง incident_id, incident_type, incident_description จาก IncidentTracking Service (หรือ Core Incident Schema) เพื่อแสดงผลให้ทีมกู้ภัยดู
- Rescue Team Info: อ้างอิง ID ของทีมกู้ภัยจากระบบจัดการทีม (เพื่อระบุว่าใครเป็นคนส่ง Log)

9. Non-Functional Requirements

NFR	รายละเอียด	สอดคล้องกับ Architecture อย่างไร
High Availability (99.9%)	ระบบต้องพร้อมใช้งาน 24/7 หยุดชะงักไม่ได้	Lambda + DynamoDB + S3 + SNS + SQS ทั้งหมดเป็น AWS Managed Services มี built-in HA หลาย AZ
Low Latency (<500ms)	Update Status และ Read Timeline ต้องตอบสนอง <500ms	Go Lambda cold start ~80-150ms, DynamoDB single-digit ms, API GW ~10-20ms → รวม ~100-200ms ปกติ
Concurrent Handling	รองรับหลายร้อยทีมพร้อมกันในช่วงวิกฤต	Lambda auto-scales (concurrent executions), DynamoDB On-Demand auto-scales (no capacity planning)
Data Integrity	Timeline ต้องเรียงลำดับเวลาถูกต้อง (Sequential Consistency)	DynamoDB Sort Key ใช้ ISO 8601 timestamp → Query ด้วย ScanIndexForward=true ได้เรียงตามเวลา
Resilience & Idempotency	รองรับ Network หลุดชั่วคราว, Client Retry ได้โดยไม่เกิด Duplicate	Client-side: เก็บ Pending Actions ใน localStorage, Retry ด้วย Idempotency Key เดิม Server-side: Lambda ตรวจสอบ Idempotency Key ใน DynamoDB ด้วย attribute_not_exists(PK) Condition → ป้องกัน Duplicate แบบ Atomic Event-level: SNS Delivery Failure → DLQ → Retry later
Data Durability	ข้อมูลต้องไม่สูญหาย	DynamoDB 99.99% durability, S3 99.99% durability, SNS + DLQ ป้องกัน Event loss

Synchronous Function Contract

Base URL: https://api.disaster-management.net/mission-progress/v1

API Contract #1: Report Progress & Update Status

ข้อมูลทั่วไป

Name	reportMissionProgress
Method	POST
Path	/incidents/{incident_id}/progress
Type	Synchronous
Lambda	report-progress (Go)

คำอธิบาย

ใช้สำหรับทีมกู้ภัยเพื่อ "บันทึกการปฏิบัติงาน" (Create new Timeline entry) และ "อัปเดตสถานะ" (Update mission status) ของภารกิจในครั้งเดียว เช่น แจ้งว่าถึงจุดเกิดเหตุแล้ว, ขอกำลังเสริม, หรือแจ้งปิดงาน พร้อมทั้ง Publish Events ไป EventBridge เพื่อแจ้ง Service อื่นๆ

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
incident_id	String	☒	รหัสเหตุการณ์ที่กำลังปฏิบัติการ (เช่น INC-001)

Headers

Header	Required	คำอธิบาย
Content-Type	☒	application/json
x-api-key	☒	API Key สำหรับ Authentication
X-Rescue-Team-ID	☒	รหัสทีมกู้ภัยที่ส่งข้อมูล (เช่น TEAM-ALPHA)

Body

{

```
"new_status": "ON_SITE",

"note": "ถึงจุดเกิดเหตุแล้ว กำลังเริ่มปฐมพยาบาลเบื้องต้น",

"new_impact_level": "HIGH"

}
```

Field	Type	Required	คำอธิบาย
new_status	String (Enum)		สถานะใหม่: EN_ROUTE, ON_SITE, NEED_BACKUP, RESOLVED
note	String		รายละเอียดการปฏิบัติงาน / หมายเหตุ
new_impact_level	String		ระดับความรุนแรงใหม่จากการประเมินหน้างาน → publish ImpactLevelUpdated event

Response

Success (200 OK)

```
{

  "message": "Progress reported successfully",

  "mission_id": "MSN-001",

  "incident_id": "INC-001",

  "old_status": "EN_ROUTE",

  "new_status": "ON_SITE",

  "updated_at": "2025-06-14T09:32:15Z"

}
```

Error (400 Bad Request) — State Transition ผิดกฎ

```
{

  "error": "INVALID_STATE_TRANSITION",

  "code": "INVALID_STATE_TRANSITION",

  "message": "Cannot transition from EN_ROUTE to RESOLVED"

}
```

```
}
```

Error (400 Bad Request) — ไม่ส่ง new_status

```
{
```

```
"error": "MISSING_PARAMETER",
```

```
"code": "MISSING_PARAMETER",
```

```
"message": "new_status is required"
```

```
}
```

Error (400 Bad Request) — สถานะไม่รู้จัก

```
{
```

```
"error": "INVALID_STATUS",
```

```
"code": "INVALID_STATUS",
```

```
"message": "Invalid status value: UNKNOWN_STATUS"
```

```
}
```

Error (400 Bad Request) — JSON body ไม่ถูกต้อง

```
{
```

```
"error": "INVALID_BODY",
```

```
"code": "INVALID_BODY",
```

```
"message": "Invalid request body"
```

```
}
```

Error (404 Not Found) — ไม่พบ incident_id

```
{
```

```
"error": "INCIDENT_NOT_FOUND",
```

```
"code": "INCIDENT_NOT_FOUND",
```

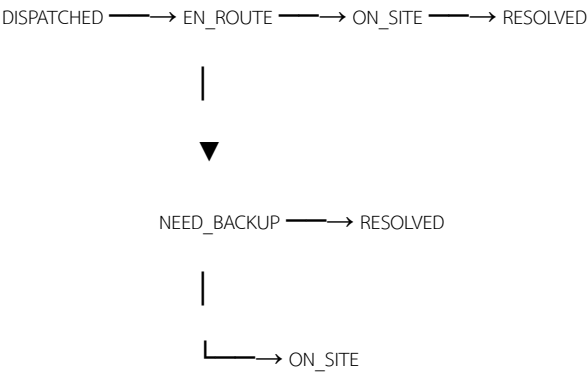
```
"message": "No mission found for incident: INC-99999"
```

```
}
```

Error (403 Forbidden) — Auth ไม่ผ่าน

```
{  
  
  "message": "Forbidden"  
  
}
```

State Machine (Validation Rules)



จาก	ไปได้	ไปไม่ได้
DISPATCHED	EN_ROUTE	❌ ข้ามขั้นตอนไม่ได้
EN_ROUTE	ON_SITE	❌ RESOLVED, NEED_BACKUP
ON_SITE	NEED_BACKUP, RESOLVED	❌ EN_ROUTE, DISPATCHED
NEED_BACKUP	ON_SITE, RESOLVED	❌ EN_ROUTE, DISPATCHED
RESOLVED	❌ (Final State)	❌ ทุกสถานะ

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Write)	อัปเดต MissionAssignment table (current_status, last_updated_at) + Insert entry ใน MissionTimeline table
EventBridge (Async)	Publish สูงสุด 3 Events: MissionStatusChanged (ทุกครั้ง), MissionBackupRequested (ถ้า NEED_BACKUP), ImpactLevelUpdated (ถ้ามี new_impact_level)
Outbox Fallback	หาก EventBridge Publish ล้มเหลว → บันทึกลง EventOutbox table → POST request ไม่ fail (ข้อมูลสถานะ safe ใน DynamoDB แล้ว)
Validation	ตรวจสอบ State Transition ตาม Business Rules ก่อนบันทึก — reject ถ้าไม่ถูกกฎ

API Contract #2: View Incident Mission Details

(Action B: ดูข้อมูลเหตุการณ์)

ข้อมูลทั่วไป

Name	getMissionDetails
Method	GET
Path	/incidents/{incident_id}
Type	Synchronous
Lambda	get-mission (Go)
Demo 1	☒ Implemented

คำอธิบาย

ใช้ดึงข้อมูลรายละเอียดของภารกิจ รวมถึง "ประวัติการทำงานทั้งหมด (Timeline)" และ "ข้อมูลเหตุการณ์จาก IncidentTracking Service" (Degraded Mode เมื่อเรียกไม่สำเร็จ) เพื่อให้ทีมกู้ภัยซ้อนหลังหรือ Dispatcher ตรวจสอบความคืบหน้า

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
incident_id	String	☒	รหัสเหตุการณ์ที่ต้องการดู (เช่น INC-001)

Headers

Header	Required	คำอธิบาย
x-api-key	☒	API Key สำหรับ Authentication
X-Rescue-Team-ID	☒	รหัสทีมกู้ภัย

Body: ไม่มี

Response

Success (200 OK) — Degraded Mode (Demo 1 เสื่อม)

```
{

  "incident_id": "INC-001",

  "mission_id": "MSN-001",

  "rescue_team_id": "TEAM-ALPHA",

  "current_status": "ON_SITE",

  "latest_impact_level": 3,

  "started_at": "2024-12-01T08:00:00Z",

  "last_updated_at": "2025-06-14T09:32:15Z",

  "timeline": [

    {

      "mission_id": "MSN-001",

      "timestamp": "2024-12-01T08:00:00Z",

      "log_id": "LOG-001",

      "action_type": "STATUS_CHANGE",

      "description": "Mission dispatched to TEAM-ALPHA",

      "performed_by": "SYSTEM"

    },

    {

      "mission_id": "MSN-001",

      "timestamp": "2025-06-14T09:32:15Z",

      "log_id": "LOG-002",

      "action_type": "STATUS_CHANGE",
```

```
"description": "ถึงจุดเกิดเหตุแล้ว",

"old_status": "EN_ROUTE",

"new_status": "ON_SITE",

"performed_by": "TEAM-ALPHA"

},

"data_source": "partial"

}
```

Success (200 OK) — Full Mode (Demo 2+ เมื่อ IncidentTracking พร้อม)

```
{

"incident_id": "INC-001",

"mission_id": "MSN-001",

"rescue_team_id": "TEAM-ALPHA",

"current_status": "ON_SITE",

"latest_impact_level": 3,

"started_at": "2024-12-01T08:00:00Z",

"last_updated_at": "2025-06-14T09:32:15Z",

"description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",

"location": "13.7563,100.5018",

"incident_type": "FLOOD",

"timeline": [ ... ],

"data_source": "full"

}
```

Field	มี Degraded	มี Full?	Source
-------	-------------	----------	--------

	?		
incident_id, mission_id, rescue_team_id	✓	✓	MissionAssignment table
current_status, latest_impact_level	✓	✓	MissionAssignment table
started_at, last_updated_at	✓	✓	MissionAssignment table
timeline	✓	✓	MissionTimeline table
description, location, incident_type	✗	✓	IncidentTracking Service
data_source	"partial"	"full"	—

Error (404 Not Found)

```
{  
  
  "error": "INCIDENT_NOT_FOUND",  
  
  "code": "INCIDENT_NOT_FOUND",  
  
  "message": "No mission found for incident: INC-99999"  
}
```

Error (403 Forbidden)

```
{  
  
  "message": "Forbidden"  
}
```

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Read)	อ่าน MissionAssignment table (state) + MissionTimeline table (timeline เรียงตาม timestamp)
External Dependency	HTTP GET → IncidentTracking Service เพื่อดึง description, location, incident_type (timeout: 3 วินาที)

Degraded Mode	IncidentTracking ล่ม/timeout → ส่งข้อมูลเฉพาะที่
---------------	--

API Contract #3: Request Presigned URL for Evidence Upload

ข้อมูลทั่วไป

Name	requestEvidenceUploadURL
Method	POST
Path	/incidents/{incident_id}/presigned-url
Type	Synchronous
Lambda	presigned-url (Go)

คำอธิบาย

ใช้สำหรับทีมกู้ภัยเพื่อ "ขอ Presigned URL" จากระบบ ก่อนอัปโหลดรูปภาพหลักฐานจากหน้างาน (Evidence Image) ตรงไปยัง Amazon S3 โดยไม่ต้องส่งไฟล์ผ่าน Lambda (ลดภาระ Server + หลีกเลียง Lambda payload limit 6MB)

Flow การใช้งาน

- 1. Frontend เรียก POST /incidents/{id}/presigned-url → ส่ง file_name + content_type
- 2. Lambda สร้าง Presigned URL (PUT) อายุ 5 นาที → ส่งกลับ upload_url + image_key
- 3. Frontend ใช้ upload_url อัปโหลดไฟล์ตรงไป S3 (HTTP PUT)
- 4. Frontend ส่ง image_key ไปพร้อม POST /incidents/{id}/progress (เพื่อเชื่อม Evidence กับ Timeline entry)

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
incident_id	String		รหัสเหตุการณ์ที่ต้องการอัปโหลดรูปหลักฐาน (เช่น INC-001)

Headers

Header	Required	คำอธิบาย
Content-Type	✓	application/json
x-api-key	✓	API Key สำหรับ Authentication
X-Rescue-Team-ID	✓	รหัสทีมกู้ภัยที่ส่งข้อมูล (เช่น TEAM-ALPHA)

Body

```
{
  "file_name": "flood-evidence-001.jpg",
  "content_type": "image/jpeg"
}
```

Field	Type	Required	คำอธิบาย
file_name	String	✓	ชื่อไฟล์ที่ต้องการอัปโหลด (เช่น photo-001.jpg)
content_type	String	✓	MIME type ของไฟล์ (เช่น image/jpeg, image/png)

Response

Success (200 OK)

```
{
  "upload_url":
    "https://s3.amazonaws.com/mission-evidence-bucket/evidence/INC-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg?X-Amz-Algorithm=AWS4-HMAC-SHA256&...",
  "image_key": "evidence/INC-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg",
  "expires_in": 300,
  "message": "Presigned URL generated successfully"
}
```

Field	Type	คำอธิบาย
upload_url	String	Presigned URL สำหรับ HTTP PUT อัปโหลดไฟล์ตรงไป S3
image_key	String	S3 Key ของไฟล์ — ใช้แนบไปกับ POST /progress เพื่อเชื่อมกับ Timeline
expires_in	Integer	อายุของ URL เป็นวินาที (300 = 5 นาที)
message	String	ข้อความยืนยัน

S3 Key Format

```
evidence/{incident_id}/{team_id}/{unix_timestamp}-{file_name}
```

ตัวอย่าง:

evidence/INC-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg

ใช้ {unix_timestamp} เพื่อป้องกันชื่อไฟล์ซ้ำ

Error (400 Bad Request) — ไม่ส่ง field ที่จำเป็น

```
{
  "error": "MISSING_PARAMETER",
  "code": "MISSING_PARAMETER",
  "message": "file_name and content_type are required"
}
```

Error (400 Bad Request) — content_type ไม่รองรับ

```
{
  "error": "INVALID_CONTENT_TYPE",
  "code": "INVALID_CONTENT_TYPE",
  "message": "Supported content types: image/jpeg, image/png, image/webp"
}
```

Error (404 Not Found) — ไม่พบ incident_id

```
{
  "error": "INCIDENT_NOT_FOUND",
  "code": "INCIDENT_NOT_FOUND",
  "message": "No mission found for incident: INC-99999"
}
```

Error (403 Forbidden)

```
{
  "message": "Forbidden"
}
```

Frontend Upload Flow (หลังได้ Presigned URL)

Frontend ใช้ upload_url ที่ได้มา อัปโหลดไฟล์ตรงไป S3

```
curl -X PUT \
```

```
-H "Content-Type: image/jpeg" \
```

```
--data-binary @flood-evidence-001.jpg \
```

```
"https://s3.amazonaws.com/mission-evidence-bucket/evidence/INC-001/...?X-Amz-Algorithm=..."
```



```
# HTTP 200 = อัปโหลดสำเร็จ
# จากนั้นส่ง image_key ไปพร้อม POST /progress
curl -X POST \
  -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  -H "Content-Type: application/json" \
  -d '{
    "new_status": "ON_SITE",
    "note": "ถึงจุดเกิดเหตุ น้ำสูง 1.2m",
    "image_key": "evidence/INC-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg"
  }' \
  "$API_URL/incidents/INC-001/progress"
```

Dependency / Reliability

Internal Data (Read)	ตรวจสอบว่า incident_id มี Mission อยู่ใน MissionAssignment table
AWS S3	สร้าง Presigned URL (PUT) ด้วย AWS SDK — ไม่ได้ upload ไฟล์จริงใน step นี้
Validation	ตรวจสอบ file_name ไม่ว่าง, content_type เป็นรูปภาพที่รองรับ (jpeg/png/webp)
Security	Presigned URL อายุ 5 นาที → หมดอายุแล้วใช้ไม่ได้ ต้องขอใหม่
Upload Failure	ถ้า Frontend อัปโหลดไป S3 ไม่สำเร็จ → อนุญาตให้ "ข้าม" (Skip) → ส่งเฉพาะ Text Status ก่อนได้

API Contract #4: List Team Missions

ข้อมูลทั่วไป

Name	listTeamMissions
Method	GET
Path	/incidents
Type	Synchronous
Lambda	get-mission (Go) — เพิ่ม handler path ใหม่ หรือ แยก Lambda ใหม่

คำอธิบาย

ใช้ดึง รายการภารกิจทั้งหมด ของทีมกู้ภัยที่ระบุ เพื่อให้ทีมกู้ภัยเห็นภาพรวมว่าตัวเองมีกี่ภารกิจ แต่ละภารกิจอยู่สถานะอะไร → กดเข้าไปดู Timeline ละเอียดด้วย
GET /incidents/{incident_id} ต่อได้

Use Cases

- ทีมกู้ภัยเปิดแอปมา → เห็นรายการภารกิจทั้งหมดของตัวเอง
- Dispatcher ดูว่าทีมหนึ่งๆ รับผิดชอบภารกิจอะไรบ้าง
- กรอง Active missions (ยังไม่ RESOLVED) เพื่อโฟกัสงานที่ต้องทำ

Request

Query Parameters

Parameter	Type	Required	คำอธิบาย
team_id	String		รหัสทีมกู้ภัย (เช่น TEAM-ALPHA)
status	String		กรองเฉพาะสถานะ (เช่น ON_SITE, NEED_BACKUP) ถ้าไม่ส่ง = ดึงทุกสถานะ

Headers

Header	Required	คำอธิบาย
x-api-key	✓	API Key สำหรับ Authentication
X-Rescue-Team-ID	✓	รหัสทีมกู้ภัย

Body: ไม่มี

ตัวอย่าง Request

```
# ดึงภารกิจทั้งหมดของ TEAM-ALPHA
curl -s -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  "$API_URL/incidents?team_id=TEAM-ALPHA" | jq .
```

```
# ดึงเฉพาะภารกิจที่สถานะ ON_SITE
curl -s -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  "$API_URL/incidents?team_id=TEAM-ALPHA&status=ON_SITE" | jq .
```

Response

Success (200 OK)

```
{
  "team_id": "TEAM-ALPHA",
  "total_missions": 3,
  "missions": [
    {
      "mission_id": "MSN-001",
      "incident_id": "INC-001",
      "current_status": "ON_SITE",
      "latest_impact_level": 3,
      "started_at": "2024-12-01T08:00:00Z",
      "last_updated_at": "2025-06-14T09:32:15Z"
    },
    {
      "mission_id": "MSN-003",
      "incident_id": "INC-003",
```

```
"current_status": "NEED_BACKUP",
"latest_impact_level": 5,
"started_at": "2025-06-14T07:00:00Z",
"last_updated_at": "2025-06-14T10:15:00Z"
},
{
  "mission_id": "MSN-005",
  "incident_id": "INC-005",
  "current_status": "RESOLVED",
  "latest_impact_level": 2,
  "started_at": "2025-06-13T14:00:00Z",
  "last_updated_at": "2025-06-13T18:30:00Z"
}
]
```

Field	Type	คำอธิบาย
team_id	String	รหัสทีมกู้ภัยที่ค้นหา
total_missions	Integer	จำนวนภารกิจทั้งหมดที่พบ
missions	Array	รายการภารกิจ (สรุป ไม่รวม Timeline)
missions[].mission_id	String	รหัสภารกิจ
missions[].incident_id	String	รหัสเหตุการณ์
missions[].current_status	String	สถานะปัจจุบัน
missions[].latest_impact_level	Integer	ระดับความรุนแรงล่าสุด
missions[].started_at	String (ISO 8601)	เวลาเริ่มภารกิจ
missions[].last_updated_at	String (ISO 8601)	เวลาอัปเดตล่าสุด

```
Success (200 OK) — ไม่พบภารกิจ
{
  "team_id": "TEAM-UNKNOWN",
  "total_missions": 0,
  "missions": []
}
```

```
Success (200 OK) — กรองด้วย status
{
  "team_id": "TEAM-ALPHA",
```

```
"total_missions": 1,
"missions": [
  {
    "mission_id": "MSN-001",
    "incident_id": "INC-001",
    "current_status": "ON_SITE",
    "latest_impact_level": 3,
    "started_at": "2024-12-01T08:00:00Z",
    "last_updated_at": "2025-06-14T09:32:15Z"
  }
]
```

Error (400 Bad Request) — ไม่ส่ง team_id

```
{
  "error": "MISSING_PARAMETER",
  "code": "MISSING_PARAMETER",
  "message": "team_id query parameter is required"
}
```

Error (400 Bad Request) — status ไม่ถูกต้อง

```
{
  "error": "INVALID_STATUS",
  "code": "INVALID_STATUS",
  "message": "Invalid status filter: UNKNOWN_STATUS. Valid values: DISPATCHED, EN_ROUTE, ON_SITE, NEED_BACKUP, RESOLVED"
}
```

Error (403 Forbidden)

```
{
  "message": "Forbidden"
}
```

DynamoDB Query Strategy

ใช้ GSI "team-index" ของ MissionAssignment table:

GSI: team-index

Partition Key: rescue_team_id = "TEAM-ALPHA"

→ ได้ภารกิจทั้งหมดของทีม

ถ้ามี status filter:

→ Filter Expression: current_status = :status

→ DynamoDB filter หลัง query (ไม่ใช่ Key Condition)

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Read)	Query MissionAssignment table ผ่าน GSI team-index (Partition Key = rescue_team_id)
No External Dependency	ไม่เรียก Service อื่น — อ่านจาก DynamoDB ของตัวเองเท่านั้น
Validation	ตรวจสอบ team_id ไม่ว่าง, status (ถ้ามี) เป็นค่าที่ถูกต้อง
Performance	GSI query = single-digit ms / Filter Expression ทำที่ DynamoDB → ไม่กระทบ Lambda
Empty Result	ไม่พบภารกิจ → return 200 OK พร้อม missions: [] (ไม่ใช่ 404)

Asynchronous Function Contract

Message Contract #1: MissionStatusChanged

ข้อมูลทั่วไป

Field	Value
Message Name	MissionStatusChangedEvent
Interaction Style	Asynchronous (Publish/Subscribe)
Producer	MissionProgress Service (report-progress Lambda — Go)
Consumers	IncidentTracking Service, Dispatch Management Service
Channel	Amazon EventBridge — Custom Event Bus: mission-progress-events
Demo 1	 Implemented → Target: CloudWatch Logs
Demo 2+	 Route ไป Service จริง

คำอธิบาย

Event นี้ถูก publish ทุกครั้งที่ทีมกู้ภัยเปลี่ยนสถานะภารกิจสำเร็จ (ผ่าน State Machine validation แล้ว) เพื่อให้:

- IncidentTracking Service — อัปเดตสถานะรวมของเหตุการณ์ (เช่น เปลี่ยน Incident เป็น "In Progress")
- Dispatch Management Service — ปลดล็อกทีมกู้ภัย BUSY → AVAILABLE (เฉพาะ RESOLVED)

Trigger: ทุกครั้งที่ POST /incidents/{id}/progress สำเร็จ

Message Format (EventBridge Event)

```
{  
  
  "source": "mission-progress-service",  
  
  "detail-type": "MissionStatusChanged",  
  
  "detail": {  
  
    "mission_id": "MSN-001",
```

```
"incident_id": "INC-001",

"rescue_team_id": "TEAM-ALPHA",

"old_status": "EN_ROUTE",

"new_status": "ON_SITE",

"note": "ถึงจุดเกิดเหตุแล้ว น้ำสูง 1.2m",

"updated_at": "2025-06-14T09:32:15Z",

"performed_by": "TEAM-ALPHA"

}

}
```

Field Definition

Field	Type	Required	Description
source	String	✓	ชื่อ Service ที่ publish (คงที่: mission-progress-service)
detail-type	String	✓	ประเภท Event (คงที่: MissionStatusChanged)
detail.mission_id	String	✓	รหัสภารกิจ
detail.incident_id	String	✓	รหัสเหตุการณ์
detail.rescue_team_id	String	✓	รหัสทีมกู้ภัยที่ปฏิบัติงาน
detail.old_status	String	✓	สถานะก่อนเปลี่ยน
detail.new_status	String	✓	สถานะใหม่ (Enum: DISPATCHED, EN_ROUTE, ON_SITE, NEED_BACKUP, RESOLVED)
detail.note	String	✗	หมายเหตุจากทีมกู้ภัย
detail.updated_at	String (ISO 8601)	✓	เวลาที่เปลี่ยนสถานะ
detail.performed_by	String	✓	ผู้กระทำ (= rescue_team_id)

Validation Rules

- new_status ต้องเป็นค่าที่ถูกต้องตาม State Machine
- updated_at ต้องเป็น ISO 8601 format
- rescue_team_id ห้ามว่าง

Consumer Routing (EventBridge Rules)

Consumer	Rule Filter	Demo 1	Demo 2+
IncidentTracking	detail-type: MissionStatusChanged	→ CloudWatch Logs	 → Lambda/SQS ของ IncidentTracking
Dispatch Mgmt	detail-type: MissionStatusChanged + detail.new_status: RESOLVED	→ CloudWatch Logs	 → Lambda/SQS ของ Dispatch

Failure Handling

- EventBridge Publish ล้มเหลว → Outbox Pattern: บันทึกลง EventOutbox table → Demo 2+ retry processor ส่งใหม่
- POST request ไม่ fail — ข้อมูลสถานะถูกบันทึกใน DynamoDB แล้ว
- EventBridge → Target ล้มเหลว → EventBridge built-in retry (24 ชั่วโมง)

หมายเหตุ: EventBridge เป็น fire-and-forget จากมุมมอง Producer — ไม่มี response กลับมา การตอบรับ/ปฏิเสธเป็นหน้าที่ของ Consumer

Message Contract #2: MissionBackupRequested

ข้อมูลทั่วไป

Field	Value
Message Name	MissionBackupRequestedEvent
Interaction Style	Asynchronous (Publish/Subscribe)
Producer	MissionProgress Service (report-progress Lambda — Go)
Consumers	Rescue Prioritization Service
Channel	Amazon EventBridge — Custom Event Bus: mission-progress-events
Demo 1	 Implemented → Target: CloudWatch Logs
Demo 2+	 Route ไป Service จริง

คำอธิบาย

Event นี้ถูก publish เมื่อทีมกู้ภัยเปลี่ยนสถานะเป็น NEED_BACKUP (ต้องการกำลังเสริม) เพื่อให้:

- Rescue Prioritization Service — รู้ว่าเคสนี้ต้องการกำลังเสริม → ปรับ Priority Score ใหม่

Trigger: POST `/incidents/{id}/progress` สำเร็จ และ `new_status = NEED_BACKUP`

Message Format (EventBridge Event)

```
{
  "source": "mission-progress-service",
  "detail-type": "MissionBackupRequested",
  "detail": {
    "mission_id": "MSN-001",
    "incident_id": "INC-001",
    "rescue_team_id": "TEAM-ALPHA",
```

```

"old_status": "ON_SITE",
"new_status": "NEED_BACKUP",
"note": "น้ำเพิ่มระดับเร็ว ต้องการเรือเพิ่ม 2 ลำ",
"updated_at": "2025-06-14T10:15:00Z",
"performed_by": "TEAM-ALPHA"
}
}

```

Field Definition

Field	Type	Required	Description
source	String	✓	คงที่: mission-progress-service
detail-type	String	✓	คงที่: MissionBackupRequested
detail.mission_id	String	✓	รหัสภารกิจ
detail.incident_id	String	✓	รหัสเหตุการณ์
detail.rescue_team_id	String	✓	รหัสทีมกู้ภัย
detail.old_status	String	✓	สถานะก่อนเปลี่ยน (= ON_SITE เสมอ ตาม State Machine)
detail.new_status	String	✓	คงที่: NEED_BACKUP
detail.note	String	✗	เหตุผลที่ต้องการกำลังเสริม
detail.updated_at	String (ISO 8601)	✓	เวลาที่ร้องขอ
detail.performed_by	String	✓	ผู้กระทำ

Validation Rules

- new_status ต้องเป็น NEED_BACKUP เท่านั้น (Event นี้ publish เฉพาะกรณีนี้)
- old_status ต้องเป็น ON_SITE (ตาม State Machine — เปลี่ยนเป็น NEED_BACKUP ได้จาก ON_SITE เท่านั้น)

Consumer Routing

Consumer	Rule Filter	Demo 1	Demo 2+

Rescue	detail-type:		
Prioritization	MissionBackupRequested	→ CloudWatch Logs	 → Lambda/SQS ของ Prioritization

Failure Handling:

- เหมือน Contract #1 — Outbox Pattern + EventBridge built-in retry
- Non-blocking — ระบบกู้ภัยหน้างานยังทำงานต่อได้แม้ Event ส่งไม่ถึง Prioritization

Message Contract #3: ImpactLevelUpdated

ข้อมูลทั่วไป

Field	Value
Message Name	ImpactLevelUpdatedEvent
Interaction Style	Asynchronous (Publish/Subscribe)
Producer	MissionProgress Service (report-progress Lambda — Go)
Consumers	IncidentTracking Service, Rescue Prioritization Service
Channel	Amazon EventBridge — Custom Event Bus: mission-progress-events
Demo 1	 Implemented → Target: CloudWatch Logs
Demo 2+	 Route ไป Service จริง

คำอธิบาย

Event นี้ถูก publish เมื่อทีมกู้ภัย ปรับระดับความรุนแรง (Impact Level) จากหน้างาน เพราะประเมินจากข้อมูลเดิมแล้วไม่ตรงกับความเป็นจริง เพื่อให้:

- IncidentTracking Service (Source of Truth) — อัปเดต Impact Level ใน Incident Master Data
- Rescue Prioritization Service — นำค่าล่าสุดไปคำนวณ Priority Score ใหม่

Trigger: POST `/incidents/{id}/progress` สำเร็จ และ มี field `new_impact_level` ใน request body

Message Format (EventBridge Event)

```
{  
  
  "source": "mission-progress-service",  
  
  "detail-type": "ImpactLevelUpdated",  
  
  "detail": {  
  
    "mission_id": "MSN-001",  
  
    "incident_id": "INC-001",
```

```
"rescue_team_id": "TEAM-ALPHA",

"new_impact_level": "HIGH",

"note": "น้ำเพิ่มระดับเร็วกว่าที่ประเมิน มีผู้สูงอายุติดอยู่ชั้น 2",

"updated_at": "2025-06-14T09:35:00Z",

"performed_by": "TEAM-ALPHA"

}

}
```

Field Definition

Field	Type	Required	Description
source	String	✓	คงที่: mission-progress-service
detail-type	String	✓	คงที่: ImpactLevelUpdated
detail.mission_id	String	✓	รหัสภารกิจ
detail.incident_id	String	✓	รหัสเหตุการณ์
detail.rescue_team_id	String	✓	รหัสทีมกู้ภัย
detail.new_impact_level	String	✓	ระดับความรุนแรงใหม่ที่ประเมินจากหน้างาน
detail.note	String	✗	เหตุผลที่ปรับค่า
detail.updated_at	String (ISO 8601)	✓	เวลาที่ปรับ
detail.performed_by	String	✓	ผู้กระทำ

Validation Rules

- new_impact_level ห้ามว่าง (ถ้า field นี้ถูกส่งมาใน POST body)
- updated_at ต้องเป็น ISO 8601 format

Consumer Routing

Consumer	Rule Filter	Demo 1	Demo 2+
----------	-------------	--------	---------

IncidentTracking	detail-type: ImpactLevelUpdated	→ CloudWatch Logs	 → Lambda/SQS ของ IncidentTracking
Rescue Prioritization	detail-type: ImpactLevelUpdated	→ CloudWatch Logs	 → Lambda/SQS ของ Prioritization

Failure Handling

- เหมือน Contract #1 — Outbox Pattern + EventBridge built-in retry
- Non-blocking — ทีมผู้รั้งยังทำงานได้แม้ Event ส่งไม่ถึง

Service Data

1) Mission Timeline Data (Owned by this service)

ข้อมูลประวัติการทำงานอย่างละเอียดของทีมกู้ภัย (Log) ที่เกิดขึ้นในแต่ละภารกิจ ใช้สำหรับสร้าง Timeline

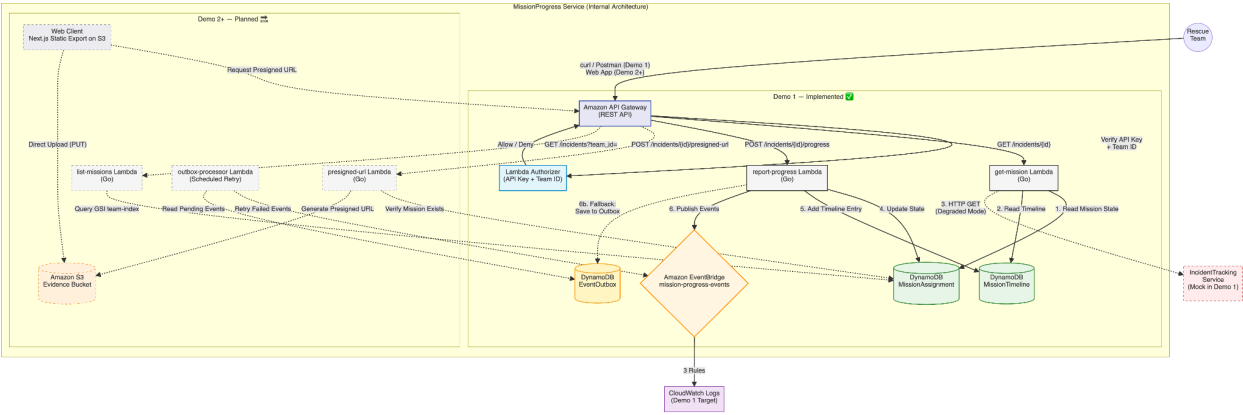
Field Name	Type	Required	Description	Example
log_id	UUID	Yes	รหัสอ้างอิงของรายการ Log (Primary Key)	LOG-556677
mission_id	UUID	Yes	รหัสภารกิจ (เชื่อมโยงกับทีมและเหตุการณ์)	MIS-998800
action_type	String	Yes	ประเภทการกระทำ เช่น "STATUS_CHANGE", "COMMENT", "UPLOAD"	STATUS_CHANGE
description	String	Yes	รายละเอียดของสิ่งที่ทำ	Arrived at location, setting up perimeter.
performed_by	String	Yes	ID ของทีมกู้ภัยหรือเจ้าหน้าที่ที่ทำรายการ	TEAM-01
timestamp	DateTime	Yes	เวลาที่เกิดเหตุการณ์จริง	2024-10-15T12:30:00Z
gps_location	String	No	พิกัด GPS ณ จุดที่บันทึกข้อมูล (Lat, Long)	13.7563, 100.5018

2) Mission Assignment State (Owned by this service)

ข้อมูลสถานะปัจจุบันของภารกิจที่ทีมกู้ภัยแต่ละทีมรับผิดชอบ (Current State)

Field Name	Type	Required	Description	Example
mission_id	UUID	Yes	รหัสภารกิจ (Primary Key)	MIS-998800
incident_id	UUID	Yes	รหัสเหตุการณ์ที่เชื่อมโยง (Foreign Key)	INC-12345
rescue_team_id	String	Yes	รหัสทีมกู้ภัยที่รับผิดชอบภารกิจนี้	TEAM-01
current_status	String	Yes	สถานะปัจจุบันของภารกิจ	ON-SITE
latest_impact_level	Integer	No	ระดับความรุนแรงล่าสุดที่ประเมินโดยทีมนี้ (1-4)	3
started_at	DateTime	Yes	เวลาที่เริ่มรับภารกิจ	2024-10-15T12:00:00Z
last_updated_at	DateTime	Yes	เวลาที่อัปเดตข้อมูลล่าสุด	2024-10-15T12:35:00Z

Service Architecture



Architecture Diagram

```
```mermaid
```

```
graph TD
```

```
%% --- External ---
```

```
User((Rescue
Team))
```

```
IncidentAPI["IncidentTracking
Service
(Mock in Demo 1)"]
```

```
ExtLog["CloudWatch Logs
(Demo 1 Target)"]
```

```
%% --- Internal Service ---
```

```
subgraph MissionProgress_Service ["MissionProgress Service (Internal Architecture)"]
```

```
subgraph Implemented ["Demo 1 — Implemented 
```

```
AGW["Amazon API Gateway
(REST API)"]
```

```
Auth["Lambda Authorizer
(API Key + Team ID)"]
```

```
GetLambda["get-mission Lambda
(Go)"]
```

```
PostLambda["report-progress Lambda
(Go)"]
```

```
DB_Assign[("DynamoDB
MissionAssignment")]
```

```
DB_Timeline[("DynamoDB
MissionTimeline")]
```

```
DB_Outbox[("DynamoDB
EventOutbox")]
```

```
EB["Amazon EventBridge
mission-progress-events"]
```

```
end
```

```
subgraph Planned ["Demo 2+ — Planned 
```

```
UI["Web Client
Next.js Static Export on S3"]
```

Storage["Amazon S3<br>Evidence Bucket"]

PresignLambda["presigned-url Lambda<br>(Go)"]

ListLambda["list-missions Lambda<br>(Go)"]

OutboxProc["outbox-processor Lambda<br>(Scheduled Retry)"]

end

end

%% --- User Flow ---

User -->|"curl / Postman (Demo 1)<br>Web App (Demo 2+)"| AGW

%% --- Auth ---

AGW -->|"Verify API Key<br>+ Team ID"| Auth

Auth -->|"Allow / Deny"| AGW

%% --- GET /incidents/{id} ---

AGW -->|"GET /incidents/{id}"| GetLambda

GetLambda -->|"1. Read Mission State"| DB\_Assign

GetLambda -->|"2. Read Timeline"| DB\_Timeline

GetLambda -->|"3. HTTP GET<br>(Degraded Mode)"| IncidentAPI

%% --- POST /incidents/{id}/progress ---

AGW -->|"POST /incidents/{id}/progress"| PostLambda

PostLambda -->|"4. Update State"| DB\_Assign

PostLambda -->|"5. Add Timeline Entry"| DB\_Timeline

PostLambda -->|"6. Publish Events"| EB

PostLambda -->|"6b. Fallback:<br>Save to Outbox"| DB\_Outbox

%% --- EventBridge ---

EB -->|"3 Rules"| ExtLog

%% --- Demo 2+: POST /incidents/{id}/presigned-url ---

AGW -->|"POST /incidents/{id}/presigned-url"| PresignLambda

PresignLambda -->|"Verify Mission Exists"| DB\_Assign

PresignLambda -->|"Generate Presigned URL"| Storage

%% --- Demo 2+: GET /incidents?team\_id= ---

AGW -->|"GET /incidents?team\_id="| ListLambda

ListLambda -->|"Query GSI team-index"| DB\_Assign

%% --- Demo 2+ Frontend flows ---

UI -->|"Request Presigned URL"| AGW

UI -->|"Direct Upload (PUT)"| Storage

%% --- Demo 2+ Outbox ---

OutboxProc -->|"Retry Failed Events"| EB

OutboxProc -->|"Read Pending Events"| DB\_Outbox

%% --- Styling ---

style AGW fill:#e8eaf6,stroke:#3f51b5,stroke-width:2px

style Auth fill:#e1f5fe,stroke:#0288d1,stroke-width:2px

```

style GetLambda fill:#f9f9f9,stroke:#333,stroke-width:2px

style PostLambda fill:#f9f9f9,stroke:#333,stroke-width:2px

style DB_Assign fill:#e8f5e9,stroke:#388e3c,stroke-width:2px

style DB_Timeline fill:#e8f5e9,stroke:#388e3c,stroke-width:2px

style DB_Outbox fill:#fff9c4,stroke:#f9a825,stroke-width:2px

style EB fill:#fff8e1,stroke:#ff8f00,stroke-width:2px

style ExtLog fill:#f3e5f5,stroke:#7b1fa2,stroke-width:1px

style IncidentAPI fill:#ffebee,stroke:#c62828,stroke-width:1px,stroke-dasharray: 5 5

style UI fill:#eceff1,stroke:#607d8b,stroke-width:1px,stroke-dasharray: 5 5

style Storage fill:#fff3e0,stroke:#f57c00,stroke-width:1px,stroke-dasharray: 5 5

style PresignLambda fill:#f9f9f9,stroke:#999,stroke-width:1px,stroke-dasharray: 5 5

style ListLambda fill:#f9f9f9,stroke:#999,stroke-width:1px,stroke-dasharray: 5 5

style OutboxProc fill:#f9f9f9,stroke:#999,stroke-width:1px,stroke-dasharray: 5 5

...

```

## Components

### 1. Amazon API Gateway (REST API):

- จุดรวมศูนย์ (Single Entry Point) รับทุก HTTP Request
- Route ไปยัง Lambda ที่ถูกต้องตาม Path + HTTP Method:
  - GET /incidents/{id} → get-mission Lambda
  - POST /incidents/{id}/progress → report-progress Lambda
  - POST /incidents/{id}/presigned-url → presigned-url
  - GET /incidents?team\_id={id} → list-missions

### 2. Lambda Authorizer (Authentication & Authorization):

- Lambda Function ที่ตั้งค่าเป็น REQUEST Authorizer ของ API Gateway

- ตรวจสอบ 2 Headers:
  - x-api-key — API Key สำหรับ Authentication
  - X-Rescue-Team-ID — ระบุตัวตนของทีมกู้ภัย
- ไม่ถูกต้อง/ไม่ส่ง → HTTP 403 Forbidden ทันที (ไม่เรียก Core Lambda → ไม่เปลืองค่าใช้จ่าย)

### 3. get-mission Lambda (Go):

- รับ GET /incidents/{incident\_id}
- อ่าน Mission State จาก MissionAssignment table
- อ่าน Timeline จาก MissionTimeline table
- เรียก IncidentTracking Service (HTTP GET) เพื่อดึงรายละเอียดเหตุการณ์:
  - สำเร็จ → data\_source: "full" (แสดงข้อมูลครบ)
  - ล้มเหลว → data\_source: "partial" (Degraded Mode — ยังใช้งานได้)
-  Demo 1: IncidentTracking ถูก Mock (URL = localhost:9999) → timeout → เข้า Degraded Mode เสมอ

### 4. report-progress Lambda (Go):

- รับ POST /incidents/{incident\_id}/progress
- ตรวจสอบ State Transition ตาม Business Rules (State Machine)
- อัปเดต State ใน MissionAssignment + เพิ่ม Log ใน MissionTimeline
- Publish Events ไป EventBridge (สูงสุด 3 events ต่อ request)
- หาก EventBridge Publish ล้มเหลว → บันทึกลง EventOutbox table (Outbox Pattern)

## 5. Amazon DynamoDB (3 Tables):

Table	PK	SK	GSIs	วัตถุประสงค์
MissionAssignment	mission_id	—	incident-index, team-index	สถานะปัจจุบันของภารกิจ
MissionTimeline	mission_id	timestamp	log-id-index	ประวัติการปฏิบัติงานเรียงตามเวลา
EventOutbox	outbox_id	created_at	status-index + TTL on ttl	Events ที่ Publish ไม่สำเร็จ (Outbox Pattern)

- ทุก Table ใช้ PAY\_PER\_REQUEST (On-Demand) → ไม่ต้อง plan capacity, auto-scale
- GSI team-index ใช้โดย list-missions Lambda เพื่อ query ภารกิจทั้งหมดของทีม

## 6. Amazon EventBridge (Custom Event Bus):

- Bus name: mission-progress-events
- report-progress Lambda publish 3 ประเภท Event:

Event	Trigger	รายละเอียด
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	สถานะเก่า → ใหม่ + note + ผู้กระทำ
MissionBackupRequested	new_status = NEED_BACKUP	ทีมกู้ภัยต้องการกำลังเสริม
ImpactLevelUpdated	มี new_impact_level ใน body	ทีมกู้ภัยปรับระดับความรุนแรงจากหน่วยงาน

- Demo 1: 3 EventBridge Rules → Target เป็น CloudWatch Logs (สำหรับ verify ว่า events ถูก publish จริง)
- Demo 2+: Rules จะ route ไปยัง Service จริง (IncidentTracking, Dispatch, Prioritization)

## 7. IncidentTracking Service (External Dependency):

- get-mission Lambda เรียก GET /incidents/{incident\_id} เพื่อดึง description, location, incident\_type
- Demo 1: Mock URL (localhost:9999) → timeout 3 วินาที → Degraded Mode



- Demo 2+: เปลี่ยน URL เป็น Service จริง (แก้ Terraform variable → apply ใหม่) → Full Mode

#### 8. [Demo 2+] Web Client (Next.js Static Export on S3):

- พัฒนาด้วย Next.js (Static Export mode — output: 'export')
- Responsive Web ให้แสดงผลดีบนมือถือ
- Hosting: S3 Static Website Hosting

#### 9. [Demo 2+] presigned-url Lambda (Go):

- รับ POST /incidents/{incident\_id}/presigned-url
- ตรวจสอบว่า incident มี Mission อยู่ใน MissionAssignment table
- Validate file\_name + content\_type (รองรับ jpeg/png/webp)
- สร้าง S3 Presigned URL (PUT) อายุ 5 นาที → Frontend อัปโหลดตรงไป S3
- S3 Key format: evidence/{incident\_id}/{team\_id}/{unix\_timestamp}-{file\_name}

#### 10. [Demo 2+] list-missions Lambda (Go):

- รับ GET /incidents?team\_id={team\_id}
- Query MissionAssignment table ผ่าน GSI team-index (Partition Key = rescue\_team\_id)
- รองรับ optional filter status (Filter Expression หลัง query)
- ส่งกลับรายการภารกิจทั้งหมดของทีม (สรุป ไม่รวม Timeline)
- ไม่พบภารกิจ → return 200 OK พร้อม missions: [] (ไม่ใช่ 404)

#### 11. [Demo 2+] S3 Evidence Bucket:

- เก็บรูปภาพหลักฐานจากหน้างาน
- Frontend อัปโหลดตรงด้วย Presigned URL จาก presigned-url Lambda

#### 12. [Demo 2+] outbox-processor Lambda (Scheduled Retry):

- Trigger โดย CloudWatch Events (ทุก 1 นาที)
- Scan EventOutbox table หา status = "PENDING" → retry publish ไป EventBridge

### Explanation

## 1. User Access & Authentication (การเข้าถึงและยืนยันตัวตน)

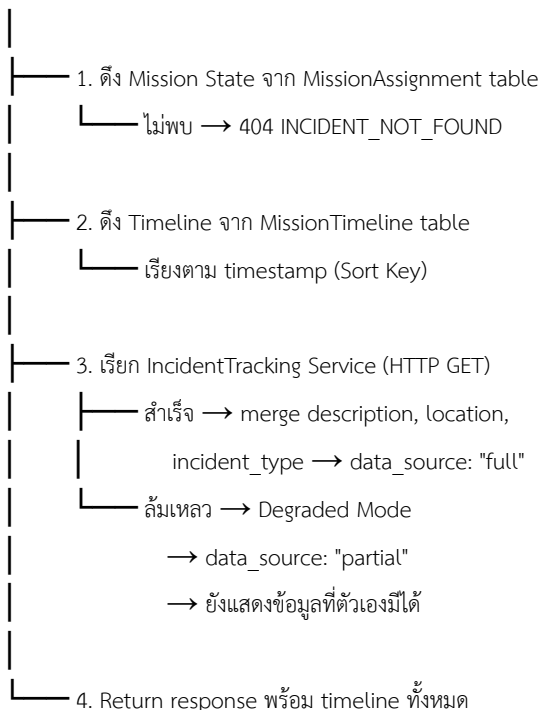
- Demo 1: ทีมกู้ภัยเข้าใช้งานผ่าน curl / Postman
- Demo 2+: เข้าใช้งานผ่าน Web Browser บนมือถือ (ไม่ต้องลงแอป)
- ทุก Request ต้องแนบ 2 Headers:
  - x-api-key — API Key สำหรับ Authentication
  - X-Rescue-Team-ID — ระบุตัวตนทีมกู้ภัย (เช่น TEAM-ALPHA)
- Lambda Authorizer ตรวจสอบทั้งสองค่า:
  -  ถูกต้อง → Route request ไปยัง Lambda ปลายทาง
  -  ไม่ถูกต้อง/ไม่ส่ง → HTTP 403 Forbidden ทันที

## 2. GET Flow — ดึงข้อมูลภารกิจ + Timeline + Incident Detail

GET /incidents/{incident\_id}



get-mission Lambda (Go)



Demo 1: Step 3 ล้มเหลวเสมอ (Mock URL) → data\_source: "partial"

Demo 2+: เปลี่ยน URL เป็น Service จริง → data\_source: "full"

### 3. POST Flow — อัปเดตสถานะภารกิจ + Publish Events

POST /incidents/{incident\_id}/progress

Body: { "new\_status", "note", "new\_impact\_level" }

|



report-progress Lambda (Go)

|

| 1. Validate State Transition (State Machine)

| | สถานะไม่รู้จัก → 400 INVALID\_STATUS

| | Transition ผิดกฎ → 400 INVALID\_STATE\_TRANSITION

| | ไม่ส่ง new\_status → 400 MISSING\_PARAMETER

|

| 2. Update MissionAssignment

| | current\_status, last\_updated\_at

|

| 3. Insert MissionTimeline entry

| | timestamp (SK), action\_type, description,

| | old/new\_status, performed\_by

|

| 4. Publish Events → EventBridge

| | MissionStatusChanged (ทุกครั้ง)

| | MissionBackupRequested (ถ้า NEED\_BACKUP)

| | ImpactLevelUpdated (ถ้ามี new\_impact\_level)

|

| 5. [Fallback] EventBridge ล้มเหลว?

└─ บันทึกลง EventOutbox table

(status: "PENDING", retry\_count: 0)

จาก	ไปได้
DISPATCHED	EN_ROUTE
EN_ROUTE	ON_SITE
ON_SITE	NEED_BACKUP, RESOLVED
NEED_BACKUP	ON_SITE, RESOLVED
RESOLVED	✗ (Final State — ไม่สามารถเปลี่ยนได้อีก)

4. Asynchronous Event Propagation (การกระจาย Event)

ทันทีที่ report-progress Lambda ประมวลผลสำเร็จ จะ publish events ไปยัง Amazon EventBridge (custom event bus: mission-progress-events):

Event	Trigger	Target (Demo 1)	Target (Demo 2+)
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	CloudWatch Logs	IncidentTracking → อัปเดตสถานะ Incident, Dispatch → ปลดล็อกทีม (RESOLVED)
MissionBackupRequested	new_status = NEED_BACKUP	CloudWatch Logs	Rescue Prioritization → คำนวณ Priority Score ใหม่
ImpactLevelUpdated	มี new_impact_level ใน body	CloudWatch Logs	IncidentTracking → อัปเดต Impact Level, Rescue Prioritization → คำนวณใหม่

EventBridge Rule ตัวอย่าง (Demo 1 → CloudWatch Logs):

```
{
 "source": ["mission-progress-service"],
 "detail-type": ["MissionStatusChanged"]
}
```

5. Reliability & Failure Handling (ความทนทาน)

ระดับ 1 — IncidentTracking ล้ม (Degraded Mode):

- get-mission Lambda เรียก IncidentTracking ไม่สำเร็จ (timeout 3 วินาที)
- ส่งข้อมูลเฉพาะที่ตัวเองมี (data\_source: "partial")
- ทีมกู้ภัยยังอัปเดตสถานะและดู Timeline ได้ตามปกติ
- ระบบ ไม่หยุดทำงาน เพราะ External Service ล้ม

ระดับ 2 — EventBridge Publish ล้มเหลว (Outbox Pattern):

```
Lambda Publish EventBridge ล้มเหลว
|
```



บันทึกใน EventOutbox Table:

```
{
 "outbox_id": "OBX-uuid",
 "created_at": "2025-06-14T09:32:16Z",
 "event_type": "MissionStatusChanged",
 "event_payload": "{...}",
 "status": "PENDING",
 "retry_count": 0,
 "ttl": 1718452335 (7 วัน)
}
```



[Demo 1] Records สะสมไว้ (ยังไม่มี processor)

[Demo 2+] outbox-processor Lambda (CloudWatch Events ทุก 1 นาที)

- Scan status = "PENDING"
- Retry publish ไป EventBridge
- สำเร็จ → status = "SENT"
- ล้มเหลวอีก → retry\_count + 1
- retry\_count > 5 → status = "FAILED" → แจ้ง Admin

สำคัญ: Request หลัก (POST) ไม่ fail เพราะ EventBridge ล้มเหลว — ข้อมูลสถานะถูกบันทึกใน DynamoDB แล้ว, Event จะถูกส่งซ้ำที่หลังผ่าน Outbox

ระดับ 3 — Lambda Authorizer ล้ม:

- API Gateway → HTTP 500 Internal Server Error
- Client (Frontend / curl) แจ้งเตือน "ระบบขัดข้อง กรุณาลองใหม่"

ระดับ 4 — [Demo 2+] Client-side Resilience:

- Web App เก็บ Pending Actions ใน localStorage เมื่อ Network หลุด
- พอ Network กลับมา → Retry อัตโนมัติ

## 6. [Demo 2+] Evidence Upload Flow — ขอ Presigned URL + อัปโหลดรูปหลักฐาน

POST /incidents/{incident\_id}/presigned-url

Body: { "file\_name", "content\_type" }



presigned-url Lambda (Go)



1. Validate input

- file\_name ว่าง → 400 MISSING\_PARAMETER
- content\_type ไม่รองรับ → 400 INVALID\_CONTENT\_TYPE

2. ตรวจสอบว่า incident มี Mission อยู่

- ไม่พบ → 404 INCIDENT\_NOT\_FOUND

3. สร้าง S3 Key

- evidence/{incident\_id}/{team\_id}/{timestamp}-{file\_name}

4. สร้าง Presigned URL (PUT) อายุ 5 นาที

5. Return { upload\_url, image\_key, expires\_in }

Frontend ได้ Presigned URL แล้ว:



6. HTTP PUT อัปโหลดไฟล์ตรงไป S3

- สำเร็จ → ได้ image\_key
- ล้มเหลว → แจ้งเตือน User  
→ อนุญาตให้ "ข้าม" ส่งแค่ Text Status

7. ส่ง image\_key ไปพร้อม

POST /incidents/{id}/progress

→ เชื่อม Evidence กับ Timeline entry

ทำไมใช้ Presigned URL?

- Lambda มี Payload limit 6MB — หากส่งรูปผ่าน Lambda จะจำกัดขนาดไฟล์
- Upload ตรงไป S3 รองรับไฟล์ได้ถึง 5GB
- ลดภาระ Lambda — ไม่ต้องรับ/ส่งไฟล์ Binary แค่สร้าง URL

## 7. [Demo 2+] List Missions Flow — ดูรายการภารกิจทั้งหมดของทีม

GET /incidents?team\_id=TEAM-ALPHA&status=ON\_SITE

|



list-missions Lambda (Go)

|

|

1. Validate input

|

team\_id ว่าง → 400 MISSING\_PARAMETER

|

status ไม่ถูกต้อง → 400 INVALID\_STATUS

|

|

2. Query DynamoDB GSI "team-index"

|

Partition Key = rescue\_team\_id

|

|

3. Apply Filter (ถ้ามี status parameter)

|

Filter Expression: current\_status = :status

|

|

4. Return { team\_id, total\_missions, missions[] }

ไม่พบภารกิจ → 200 OK + missions: []

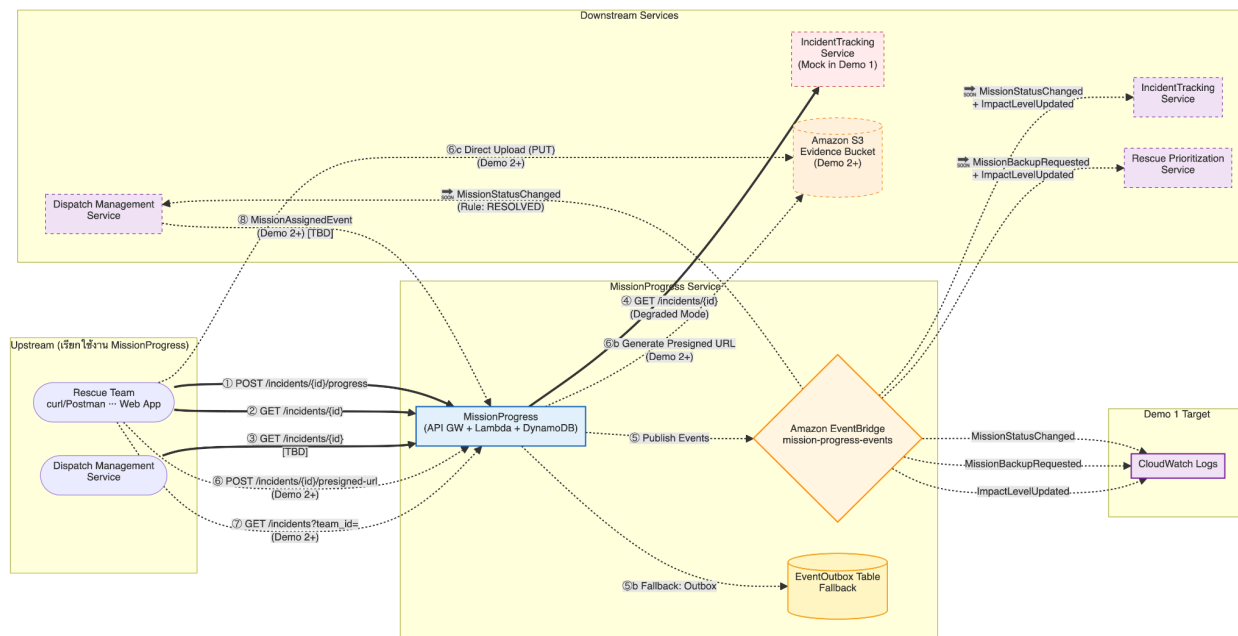
Use Cases:

- ทีมกู้ภัยเปิดแอป → เห็นรายการภารกิจทั้งหมดของตัวเอง
- Dispatcher ดูว่าทีมหนึ่งๆ รับผิดชอบอะไรบ้าง
- กดเข้าไปดู Timeline ละเอียดด้วย GET /incidents/{id} ต่อได้



## # Service Interaction

\*\*Interaction Diagram\*\* \*(แสดงการสื่อสารระหว่าง MissionProgress กับ Service อื่นๆ ในระบบรวม)\*



## คำอธิบาย Flow

สัญลักษณ์	ความหมาย
เส้นทึบ (==)	Synchronous — รอผลตอบกลับทันที (HTTP Request/Response)
เส้นประ (- - -)	Asynchronous — ส่งแล้วไม่รอผล (Event ผ่าน EventBridge)

```
```mermaid
```

```
graph LR
```

```
%% --- Upstream ---
```

```
subgraph Upstream ["Upstream (เรียกใช้งาน MissionProgress)"]
```

```
    App(["Rescue Team<br>curl/Postman ... Web App"])
```

```
    DispatchUI(["Dispatch Management<br>Service"])
```

```
end
```

```
%% --- Our Service ---
```

```
subgraph Our_Service ["MissionProgress Service"]
```

```
    MS["MissionProgress<br>(API GW + Lambda + DynamoDB)"]
```

```
    EB["Amazon EventBridge<br>mission-progress-events"]
```

```
    Outbox(["EventOutbox Table<br>Fallback"])
```

```
end
```

```
%% --- Downstream ---
```

```
subgraph Downstream ["Downstream Services"]
```

```
    IncidentAPI(["IncidentTracking<br>Service<br>(Mock in Demo 1)"])
```

```
    S3(["Amazon S3<br>Evidence Bucket<br>(Demo 2+)"])
```

```
    Incident(["IncidentTracking<br>Service"])
```

```
    Dispatch(["Dispatch Management<br>Service"])
```

```
    Priority(["Rescue Prioritization<br>Service"])
```

```
end
```

```
%% --- Demo 1 Target ---
```

```
subgraph Demo1Target ["Demo 1 Target"]
```

```
    CWL["CloudWatch Logs"]
```

```
end
```

```
%% === Inbound Synchronous ===
```

```
App == "❶ POST /incidents/{id}/progress" ==> MS
```

```
App == "❷ GET /incidents/{id}" ==> MS
```

```
DispatchUI == "❸ GET /incidents/{id}<br>[TBD]" ==> MS
```

```
App -. "❹ POST /incidents/{id}/presigned-url<br>(Demo 2+)" .-> MS
```

```
App -. "❺ GET /incidents?team_id=<br>(Demo 2+)" .-> MS
```

%% === Inbound Async ===

Dispatch -. "⑧ MissionAssignedEvent
(Demo 2+) [TBD]" .-> MS

%% === Downstream Synchronous ===

MS == "④ GET /incidents/{id}
(Degraded Mode)" ==> IncidentAPI

MS -. "⑥b Generate Presigned URL
(Demo 2+)" .-> S3

%% === Downstream Asynchronous ===

MS -. "⑤ Publish Events" .-> EB

MS -. "⑤b Fallback: Outbox" .-> Outbox

%% === Frontend Direct Upload ===

App -. "⑥c Direct Upload (PUT)
(Demo 2+)" .-> S3

%% === Demo 1: EventBridge → CloudWatch Logs ===

EB -. "MissionStatusChanged" .-> CWL

EB -. "MissionBackupRequested" .-> CWL

EB -. "ImpactLevelUpdated" .-> CWL

%% === Demo 2+: EventBridge → Real Services ===

EB -. "🔴 MissionStatusChanged
+ ImpactLevelUpdated" .-> Incident

EB -. "🔴 MissionStatusChanged
(Rule: RESOLVED)" .-> Dispatch

EB -. "🔴 MissionBackupRequested
+ ImpactLevelUpdated" .-> Priority

%% --- Styling ---

style MS fill:#e3f2fd,stroke:#1565c0,stroke-width:2px

style EB fill:#fff8e1,stroke:#ff8f00,stroke-width:2px

style Outbox fill:#fff9c4,stroke:#f9a825,stroke-width:2px

style IncidentAPI fill:#ffebee,stroke:#c62828,stroke-width:1px,stroke-dasharray: 5 5

style S3 fill:#fff3e0,stroke:#f57c00,stroke-width:1px,stroke-dasharray: 5 5

style CWL fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px

style Incident fill:#f3e5f5,stroke:#7b1fa2,stroke-width:1px,stroke-dasharray: 5 5

style Dispatch fill:#f3e5f5,stroke:#7b1fa2,stroke-width:1px,stroke-dasharray: 5 5

style Priority fill:#f3e5f5,stroke:#7b1fa2,stroke-width:1px,stroke-dasharray: 5 5

...

คำอธิบาย Flow

สัญลักษณ์	ความหมาย
เส้นทึบ (==)	Synchronous — รอผลตอบกลับทันที (HTTP Request/Response)
เส้นประ (- - -)	Asynchronous — ส่งแล้วไม่รอผล (Event ผ่าน EventBridge)

Upstream Services (บริการที่เรียกใช้งาน MissionProgress)

① POST /incidents/{incident_id}/progress — Rescue Team

Purpose: รายงานสถานะล่าสุดจากหน้างาน

Method: POST

Auth: x-api-key + X-Rescue-Team-ID

Lambda: report-progress (Go)

Client: curl/Postman (Demo 1) → Web App (Demo 2+)

Demo 1:  Implemented

Request Body:

```
{  
  
  "new_status": "ON_SITE",  
  
  "note": "ถึงจุดเกิดเหตุแล้ว น้ำสูง 1.2m",  
  
  "new_impact_level": "HIGH"    // optional  
}
```

Response 200:

```
{  
  
  "message": "Progress reported successfully",  
  
  "mission_id": "MSN-001",  
  
  "incident_id": "INC-001",  
  
  "old_status": "EN_ROUTE",  
  
  "new_status": "ON_SITE",  
  
  "updated_at": "2025-..."
```

```
}
```

② GET /incidents/{incident_id} — Rescue Team

Purpose: ดึง Timeline + สถานะปัจจุบัน

Method: GET

Auth: x-api-key + X-Rescue-Team-ID

Lambda: get-mission (Go)

Demo 1:  Implemented

Response 200:

```
{  
  
  "incident_id": "INC-001",  
  
  "mission_id": "MSN-001",  
  
  "rescue_team_id": "TEAM-ALPHA",  
  
  "current_status": "ON_SITE",  
  
  "latest_impact_level": 3,  
  
  "started_at": "2024-12-01T08:00:00Z",  
  
  "last_updated_at": "2025-06-14T09:32:15Z",  
  
  "timeline": [ ... ],  
  
  "data_source": "partial"  
}
```

- data_source: "full" — เรียก IncidentTracking สำเร็จ (มี description, location, incident_type)
- data_source: "partial" — เรียกไม่สำเร็จ (Degraded Mode — Demo 1 เป็น partial เสมอ)

③ GET /incidents/{incident_id} — Dispatch Management Service

Purpose: Dispatcher ดู Timeline ละเอียด + รูปภาพหลักฐาน

Method: GET (ใช้ endpoint เดียวกับ ②)

Status: [TBD: Pending Discussion กับ Noppakron]

ทำไม Dispatch อาจต้องเรียก GET API (ไม่ใช่แค่ฟัง Event)

ข้อมูลที่ Dispatcher ต้องการ	ได้จาก Event?	ได้จาก GET API?
สถานะปัจจุบัน	✓	✓
Timeline ทั้งหมดย้อนหลัง	✗ (Event ส่งแค่ entry ล่าสุด)	✓
รูปภาพหลักฐาน (Demo 2+)	✗	✓

⑥ POST /incidents/{incident_id}/presigned-url — Rescue Team (Demo 2+)

Purpose: ขอ Presigned URL สำหรับอัปโหลดรูปหลักฐาน

Method: POST

Auth: x-api-key + X-Rescue-Team-ID

Lambda: presigned-url (Go)

Demo 1: ✗ ยังไม่ implement

Demo 2+:  Planned

Request Body:

```
{  
  
  "file_name": "flood-evidence-001.jpg",  
  
  "content_type": "image/jpeg"
```

```
}
```

Response 200:

```
{  
  
  "upload_url": "https://s3.amazonaws.com/...",  
  
  "image_key": "evidence/INC-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg",  
  
  "expires_in": 300,  
  
  "message": "Presigned URL generated successfully"  
}
```

Full Upload Flow

⑥a. Frontend → POST /incidents/{id}/presigned-url → Lambda

⑥b. Lambda → S3 (Generate Presigned URL) → Return URL + image_key

⑥c. Frontend → S3 (Direct PUT Upload ด้วย Presigned URL)

⑥d. Frontend → POST /incidents/{id}/progress (แนบ image_key) → เชื่อม Evidence กับ Timeline entry

⑦ GET /incidents?team_id={team_id} — Rescue Team (Demo 2+)

Purpose: ดึงรายการภารกิจทั้งหมดของทีม

Method: GET

Auth: x-api-key + X-Rescue-Team-ID

Lambda: list-missions (Go)

Demo 1: ❌ ยังไม่ implement

Demo 2+:  Planned

Query Parameters:

team_id (required): รหัสทีมกู้ภัย เช่น TEAM-ALPHA

status (optional): กรอง เช่น ON_SITE, NEED_BACKUP

Response 200:

```
{
  "team_id": "TEAM-ALPHA",
  "total_missions": 3,
  "missions": [
    {
      "mission_id": "MSN-001",
      "incident_id": "INC-001",
      "current_status": "ON_SITE",
      "latest_impact_level": 3,
      "started_at": "2024-12-01T08:00:00Z",
      "last_updated_at": "2025-06-14T09:32:15Z"
    },
    ...
  ]
}
```

⑧ MissionAssignedEvent — จาก Dispatch (Inbound Async, Demo 2+)

Source: Dispatch Management Service

Trigger: เมื่อ Dispatcher มอบหมายภารกิจให้ทีมกู้ภัย

Channel: EventBridge

Status: [TBD: Pending Discussion กับ Noppakron]

Demo 1: ❌ ใช้ Seed Data แทน (script/seed-data.sh)

Demo 2+:  รับ Event จาก Dispatch → สร้าง Mission Record อัตโนมัติ

Expected Payload:

```
{  
  
  "source": "dispatch-management-service",  
  
  "detail-type": "MissionAssignedEvent",  
  
  "detail": {  
  
    "mission_id": "MSN-001",  
  
    "rescue_unit_id": "TEAM-ALPHA",  
  
    "incident_id": "INC-001",  
  
    "incident_type": "FLOOD",  
  
    "incident_description": "น้ำท่วมหนัก บ้าน 2 ชั้น",  
  
    "incident_location": "13.7563,100.5018",  
  
    "impact_level": "MODERATE",  
  
    "priority": "MEDIUM",  
  
    "assigned_at": "2025-06-14T08:45:00Z"  
  
  }  
}
```

MissionProgress จะทำอะไร:

1. สร้าง MissionAssignment record (status = DISPATCHED)
2. สร้าง MissionTimeline entry แรก

3. เก็บ incident data เป็น Reference Copy

Downstream Services (บริการปลายทางที่ MissionProgress เรียกหรือส่งข้อมูลไป)

MissionProgress สื่อสารกับ Downstream ผ่าน 2 ช่องทาง:

- Synchronous (HTTP) — เรียกดึงข้อมูลแบบรอผลตอบกลับ
- Asynchronous (EventBridge Events) — ส่ง Event แล้วไม่รอผล

1. IncidentTracking Service (Owner: Krittamet Damthongkam)

บทบาท: เป็น Source of Truth ของข้อมูลเหตุการณ์ (Incident Master Data)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด	Demo 1	Demo 2+
④	Sync GET/incidents/{id}	MissionProgress → IncidentTracking	ดึง description, location, incident_type มาแสดงให้ทีมกู้ภัย (Degraded Mode เมื่อล้ม เหลว → data_source: "partial")	⚠ Mock (localhost:99 99 → timeout)	✅ URL จริง [TBD]
⑤a	AsyncMissionStatusCh anged	MissionProgress → IncidentTracking	อัปเดตสถานะรวมของ Incident (เช่น เปลี่ยน เป็น "In Progress")	→ CloudWatch Logs	➡️ SOON → Service จริง [TBD]
⑤a	AsyncImpactLevelUp dated	MissionProgress → IncidentTracking	ส่ง Impact Level ล่าสุดจากหน่วยงาน → IncidentTracking อัปเดต Master Data	→ CloudWatch Logs	➡️ SOON → Service จริง [TBD]

Expected API Response (Sync GET)

```
{  
  "incident_id": "INC-001",  
  "description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",  
  "location": "13.7563,100.5018",
```

```
"incident_type": "FLOOD"
}
```

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว (timeout 3 วินาที / error)	Degraded Mode — ส่งข้อมูลเฉพาะที่ตัวเองมี ทีมกู้ภัยยังทำงานได้ปกติ
Async Event Publish ล้มเหลว	Outbox Pattern → บันทึกลง EventOutbox table → Demo 2+ retry processor ส่งใหม่

2. Dispatch Management Service (Owner: Noppakron Songkroh)

บทบาท: จัดการการมอบหมายงานและสถานะทีมกู้ภัย (BUSY/AVAILABLE)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด	Demo 1	Demo 2+
⑤b	AsyncMissionStatusChanged(Rule: RESOLVED)	MissionProgress → Dispatch	แจ้งว่าภารกิจเสร็จ → ปลด ล็อกทีมกู้ภัย BUSY → AVAILABLE	→ CloudWatch Logs	 → Service จริง [TBD]

EventBridge Rule Filter

```
{
  "source": ["mission-progress-service"],
  "detail-type": ["MissionStatusChanged"],
  "detail": {
    "new_status": ["RESOLVED"]
  }
}
```

}

หมายเหตุ: Dispatch ยังเป็น Upstream ด้วย — ส่ง `MissionAssignedEvent` เข้ามา (⑧) + อาจเรียก GET API (③)

Failure Handling

กรณี	การจัดการ
EventBridge Publish ล้มเหลว	Outbox Pattern → EventOutbox table → retry ใน Demo 2+
EventBridge → Dispatch Target ล้มเหลว	EventBridge built-in retry (24 ชั่วโมง)

3. Rescue Prioritization Service (Owner: Nattasak Chonmanat)

บทบาท: จัดลำดับความสำคัญและความเร่งด่วนของแต่ละเคส (Priority Score)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด	Demo 1	Demo 2+
⑤c	AsyncMissionBackupRequested	MissionProgress → Prioritization	ทีมกู้ภัยต้องการกำลังเสริม (NEED_BACKUP) → คำนวณ Priority Score ใหม่	→ CloudWatch Logs	 → Service จริง [TBD]
⑤c	AsyncImpactLevelUpdated	MissionProgress → Prioritization	ส่ง Impact Level ล่าสุดจากหน้างาน → คำนวณลำดับความสำคัญใหม่	→ CloudWatch Logs	 → Service จริง [TBD]

Failure Handling

กรณี	การจัดการ
EventBridge Publish ล้มเหลว	Outbox Pattern (safety net)
Event ส่งไม่ถึง Prioritization	Non-blocking — ระบบกู้ภัยยังทำงานต่อได้ (Prioritization ไม่ใช่ Critical Path)

4. Amazon S3 — Evidence Bucket (Demo 2+)

บทบาท: เก็บรูปภาพหลักฐานจากพนักงาน

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด	Demo 1	Demo 2+
⑥b	Sync Generate Presigned URL	presigned-url Lambda → S3	สร้าง Presigned URL (PUT) อายุ 5 นาที	✗	
⑥c	Frontend → S3 Direct	Rescue Team → S3	อัปโหลดรูปตรงไป S3 ด้วย Presigned URL (ไม่ผ่าน MissionProgress)	✗	

Failure Handling

กรณี	การจัดการ
Presigned URL Generation ล้มเหลว	Lambda return 500 → User Retry
Upload ไป S3 ล้มเหลว	อนุญาตให้ "ข้าม" → ส่งเฉพาะ Text Status ก่อน

EventBridge Events — Routing Detail

⑤ Lambda → EventBridge (mission-progress-events)

report-progress Lambda publish events เมื่อ POST สำเร็จ

Event	Trigger	Payload สำคัญ
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	mission_id, incident_id, rescue_team_id, old_status, new_status, note, updated_at, performed_by
MissionBackupRequested	new_status = NEED_BACKUP	(เหมือน MissionStatusChanged)
ImpactLevelUpdated	มี new_impact_level ใน body	mission_id, incident_id, rescue_team_id, new_impact_level, note, updated_at

EventBridge Event Payload ตัวอย่าง

```
{  
  
  "source": "mission-progress-service",  
  
  "detail-type": "MissionStatusChanged",  
  
  "detail": {  
  
    "mission_id": "MSN-001",  
  
    "incident_id": "INC-001",  
  
    "rescue_team_id": "TEAM-ALPHA",  
  
    "old_status": "EN_ROUTE",  
  
    "new_status": "ON_SITE",
```


"note": "ถึงจุดเกิดเหตุแล้ว",

"updated_at": "2025-06-14T09:32:15Z",

"performed_by": "TEAM-ALPHA"

}

}

{

"source": "mission-progress-service",

"detail-type": "ImpactLevelUpdated",

"detail": {

"mission_id": "MSN-001",

"incident_id": "INC-001",

"rescue_team_id": "TEAM-ALPHA",

"new_impact_level": "HIGH",

"note": "น้ำเพิ่มระดับเร็วกว่าที่ประเมิน",

"updated_at": "2025-06-14T09:35:00Z"

}

}

Routing — Demo 1 vs Demo 2+

Demo 1 (ปัจจุบัน): 3 EventBridge Rules → CloudWatch Logs

Rule	Pattern	Target
mission-status-changed-rule	detail-type: MissionStatusChanged	CloudWatch Log Group
backup-requested-rule	detail-type: MissionBackupRequested	CloudWatch Log Group
impact-level-updated-rule	detail-type: ImpactLevelUpdated	CloudWatch Log Group

Demo 2+ (แผน): เพิ่ม Rules → Route ไป Service จริง

Event	Subscriber	EventBridge Rule Filter	Target
MissionStatusChanged	IncidentTracking	detail-type: MissionStatusChanged	Lambda / SQS ของ IncidentTracking
MissionStatusChanged	Dispatch Mgmt	detail-type: MissionStatusChanged + detail.new_status: RESOLVED	Lambda / SQS ของ Dispatch
MissionBackupRequested	Rescue Prioritization	detail-type: MissionBackupRequested	Lambda / SQS ของ Prioritization
ImpactLevelUpdated	IncidentTracking	detail-type: ImpactLevelUpdated	Lambda / SQS ของ IncidentTracking
ImpactLevelUpdated	Rescue Prioritization	detail-type: ImpactLevelUpdated	Lambda / SQS ของ Prioritization

⑤b Fallback: Outbox Pattern

EventBridge Publish ล้มเหลว

|



บันทึกลง EventOutbox Table (DynamoDB)

```
{  
  
  "outbox_id": "OBX-uuid",  
  
  "event_type": "MissionStatusChanged",  
  
  "event_payload": "{...}",  
  
  "status": "PENDING",  
  
  "retry_count": 0,  
  
  "ttl": "<7 days">  
  
}
```

Demo 1: Records สะสมไว้ (ยังไม่มี processor)

Demo 2+: outbox-processor Lambda (ทุก 1 นาที) → Scan PENDING → Retry → SENT / FAILED

สำคัญ: POST request ไม่ fail เพราะ EventBridge ล้มเหลว — ข้อมูลสถานะถูกบันทึกใน DynamoDB แล้ว

สรุป Interaction ทั้งหมด

Inbound (เข้า MissionProgress)

#	Source	ช่องทาง	Endpoint / Event	Demo 1	Demo 2+
①	Rescue Team	Sync POST	POST /incidents/{id}/progress	✓ curl/Postman	✓ Web App

②	Rescue Team	Sync GET	GET /incidents/{id}	✓	✓
③	Dispatch Mgmt	Sync GET	GET /incidents/{id}	[TBD]	[TBD]
⑥	Rescue Team	Sync POST	POST /incidents/{id}/presigned-url	✗	→ SOON
⑦	Rescue Team	Sync GET	GET /incidents?team_id={id}	✗	→ SOON
⑧	Dispatch Mgmt	Async Event	MissionAssignedEvent	✗ Seed Data	→ SOON [TBD]

Downstream / Outbound (ออกจาก MissionProgress)

#	Destination	ช่องทาง	Event / API	Demo 1	Demo 2+
④	IncidentTracking	Sync GET	GET /incidents/{id} (Degraded Mode)	⚠ Mock	✓ URL จริง
⑥b	Amazon S3	Sync	Generate Presigned URL	✗	→ SOON
⑤a	IncidentTracking	Async Event	MissionStatusChanged + ImpactLevelUpdated	→ CW Logs	→ SOON → Service จริง
⑤b	Dispatch Mgmt	Async Event	MissionStatusChanged (Rule: RESOLVED)	→ CW Logs	→ SOON → Service จริง
⑤c	Rescue Prioritization	Async Event	MissionBackupRequested + ImpactLevelUpdated	→ CW Logs	→ SOON → Service จริง

Frontend → S3 Direct (ไม่ผ่าน MissionProgress)

#	Source	Destination	ช่องทาง	Demo 1	Demo 2+
⑥c	Rescue Team (Frontend)	Amazon S3	HTTP PUT (Presigned URL)	✗	→ SOON

Downstream Services สรุป (≥2 services ของเพื่อนร่วมชั้น )

#	Service	Owner	ช่องทาง	Interaction
1	IncidentTracking Service	Krittamet Damthongkam	HTTP GET + EventBridge	Sync (ดึงข้อมูล) + Async (2 Events)
2	Dispatch Management Service	Noppakron Songkroh	EventBridge + HTTP GET	Async (RESOLVED Event) + Sync GET [TBD] + Inbound Event [TBD]
3	Rescue Prioritization Service	Nattasak Chonmanat	EventBridge	Async (2 Events)

Failure Handling

กรณี	การจัดการ	Demo 1	Demo 2+
IncidentTracking ล้ม	Degraded Mode (data_source: "partial")	 เป็น Degraded เสมอ	
EventBridge Publish ล้มเหลว	Outbox Pattern → EventOutbox table	 save only	 save + retry
Lambda Authorizer ล้ม	HTTP 500		
S3 Presigned URL ล้มเหลว	Lambda return 500 → User Retry	—	 SOON
S3 Upload ล้มเหลว	อนุญาตให้ "ข้าม" → ส่งแค่ Text Status	—	 SOON

Dependency 1: IncidentTracking Service

Field	Value
Service Owner	Krittamet Damthongkam
Type	Service
Interaction Style	Hybrid (Synchronous + Asynchronous)
Criticality	High (แต่ระบบทำงานต่อได้ใน Degraded Mode)

Purpose

ทิศทาง	ช่องทาง	รายละเอียด	Demo 1	Demo 2+
ขาออก Sync	HTTP GET /incidents/{id}	ดึง description, location, incident_type มาแสดงให้ทีมกู้ภัย (Degraded Mode เมื่อล้มเหลว)	 Mock (localhost:9999 → timeout → partial)	 URL จริง [TBD: รอ URL จาก Krittamet]
ขาออก Async	EventBridge MissionStatusChange d	อัปเดตสถานะรวม Incident (เช่น เปลี่ยน เป็น "In Progress")	 → CloudWatch Logs	 → Service จริง [TBD]
ขาออก Async	EventBridge ImpactLevelUpdate d	ส่ง Impact Level ล่าสุดจากหน่วยงาน เพื่อให้ IncidentTracking (Source of Truth) อัปเดต	 → CloudWatch Logs	 → Service จริง [TBD]

Failure Handling

กรณี	การจัดการ
Sync — เรียก GET ไม่สำเร็จ	Degraded Mode — ส่งข้อมูลเฉพาะที่ตัวเองมี (data_source: "partial") ทีมกู้ภัยยังทำงานได้ปกติ
Async — EventBridge Publish ล้มเหลว	Outbox Pattern → บันทึกลง EventOutbox table → Demo 2+ retry processor ส่งใหม่

[TBD — Pending Discussion กับ Krittamet]:

- *API path & response format* ตรงไหม?
- *Service URL* คืออะไร? (deploy แล้วหรือยัง?)
- สนใจรับ Event *MissionStatusChanged* + *ImpactLevelUpdated* ผ่าน EventBridge bus *mission-progress-events* ไหม?

Dependency 2: Dispatch Management Service

Field	Value
Service Owner	Noppakron Songkroh
Type	Service
Interaction Style	Bidirectional Asynchronous + Synchronous (GET)
Criticality	Critical

Purpose

ทิศทาง	ช่องทาง	Event / API	รายละเอียด	Demo 1	Demo 2+
ขาเข้า Async	EventBridge	MissionAssignedEvent	Dispatch มอบหมายภารกิจ → MissionProgress สร้าง Mission Record + เก็บ Incident Reference	 ใช้ Seed Data	 [TBD]
ขาออก Async	EventBridge MissionStatusChange d	Rule: detail.new_status = RESOLVED	แจ้ง Dispatch ว่าภารกิจเสร็จ → ปลด ล็อกทีมกู้ภัย BUSY → AVAILABLE	 → CloudWatch Logs	 → Service จริง [TBD]
ขาเข้า Sync	GET	GET /incidents/{id}	Dispatcher ดู Timeline ละเอียด + รูปภาพหลักฐาน	[TBD]	[TBD]

Failure Handling

กรณี	การจัดการ
ขาเข้า — ไม่ได้รับ MissionAssignedEvent	Demo 1: ไม่กระทบ (ใช้ Seed Data) / Demo 2+: Mission จะไม่ถูกสร้าง [TBD: Discuss Retry mechanism กับ Noppakron]
ขาออก — EventBridge Publish ล้มเหลว	Outbox Pattern → EventOutbox table → retry ใน Demo 2+

[TBD — Pending Discussion กับ Noppakron]:

- *MissionAssignedEvent* — คุณ publish event นี้ได้ไหม? ผ่าน EventBridge bus ไหน? Payload มี field อะไรบ้าง?
- *mission_id* ใครเป็นคน generate?
- สนใจรับ *MissionStatusChanged (RESOLVED)* ผ่าน bus *mission-progress-events* ไหม?
- ต้องการเรียก GET API ของเราไหม?

Dependency 3: Rescue Prioritization Service

Field	Value
Service Owner	Nattasak Chonmanat
Type	Service
Interaction Style	Asynchronous (Event-Driven)
Criticality	Medium (Non-blocking for rescue operation)

Purpose

ทิศทาง	Event	รายละเอียด	Demo 1	Demo 2+
ขาออก Async	EventBridge MissionBackupRequested	ทีมกู้ภัยต้องการกำลังเสริม → คำนวณ Priority Score ใหม่	✓ → CloudWatch Logs	→ Service จริง [TBD]
ขาออก Async	EventBridge ImpactLevelUpdated	ส่ง Impact Level ล่าสุด จากหน้างาน → คำนวณ ลำดับความสำคัญใหม่	✓ → CloudWatch Logs	→ Service จริง [TBD]

Failure Handling:

- Rescue Prioritization ไม่ใช่ Critical Path — ถ้า Event ส่งไม่ถึง ระบบกู้ภัยหน้างานยังทำงานได้ปกติ
- อย่างไรก็ตาม EventBridge มี built-in Retry:
 - EventBridge พยายามส่งไป Target ซ้ำอัตโนมัติ (24 ชั่วโมง)
 - หาก Lambda ของเรา Publish ไม่สำเร็จ → Outbox Pattern เป็น safety net
- Non-blocking with EventBridge built-in Retry + Outbox safety net

[TBD — Pending Discussion กับ Nattasak]:

- สนใจรับ MissionBackupRequested + ImpactLevelUpdated ผ่าน EventBridge bus mission-progress-events ใหม่?
- new_impact_level ส่งเป็น String ("HIGH") — OK ใหม่? หรือต้องการเป็นตัวเลข?

Dependency 4: Amazon S3 (Evidence Bucket)

Field	Value
Type	Infrastructure — Object Storage
Interaction Style	Synchronous
Criticality	Medium
Learner Lab	✅ ใช้ได้ + Free Tier 5GB
Demo 1	❌ ยังไม่ implement
Demo 2+	✅ presigned-url Lambda + S3 bucket

Purpose

- เก็บรูปภาพหลักฐานจากหน้างาน (Evidence Images)
- Frontend อัปโหลดตรงไป S3 ผ่าน Presigned URL

Failure Handling

กรณี	การจัดการ
Presigned URL Generation ล้มเหลว	Lambda return HTTP 500 → User กด Retry
Upload ไป S3 ล้มเหลว	อนุญาตให้ "ข้าม" (Skip) → ส่งเฉพาะ Text Status ก่อน
S3 ล่มทั้ง Service	โอกาสต่ำมาก (99.99% durability) → Degrade ได้โดยส่งแค่ Text

Dependency 5: API Gateway + Lambda Authorizer

Field	Value
Type	Infrastructure — Authentication & Authorization
Interaction Style	Synchronous
Criticality	Critical
Learner Lab	 ใช้ได้ + Free Tier
Demo 1	 Implemented

Purpose

- API Gateway (REST API): จุบรวมศูนย์รับทุก HTTP Request
- Lambda Authorizer (REQUEST type): ตรวจสอบ 2 Headers:
 - x-api-key — API Key สำหรับ Authentication
 - X-Rescue-Team-ID — ระบุตัวตนทีมกู้ภัย

Failure Handling

กรณี	การจัดการ
API Key ไม่ถูกต้อง / ไม่มี	HTTP 403 Forbidden (ไม่เรียก Core Lambda)
ไม่ส่ง X-Rescue-Team-ID	HTTP 403 "User is not authorized"
Lambda Authorizer ล่ม	HTTP 500 → Client แจ้งเตือน
API Gateway ล่มทั้ง Service	โอกาสต่ำมาก (AWS Managed, Multi-AZ) → Client เก็บ Pending Actions ใน localStorage (Demo 2+)

Dependency 6: Amazon EventBridge (Custom Event Bus)

Field	Value
Type	Infrastructure — Event Bus
Bus Name	mission-progress-events
Interaction Style	Asynchronous
Criticality	Critical
Learner Lab	✅ ใช้ได้ (deploy สำเร็จแล้ว)
Demo 1	✅ Implemented (3 Rules → CloudWatch Logs)

Purpose

- รับ Events จาก report-progress Lambda แล้ว Route ไปยัง Target ตาม Rules
- Demo 1: Target = CloudWatch Logs (verify events)
- Demo 2+: Target = Service จริง (IncidentTracking, Dispatch, Prioritization)

Events ที่ Publish

Event	Trigger	Target (Demo 1)	Target (Demo 2+)
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	CloudWatch Logs	IncidentTracking, Dispatch (Rule: RESOLVED)
MissionBackupRequested	new_status = NEED_BACKUP	CloudWatch Logs	Rescue Prioritization
ImpactLevelUpdated	มี new_impact_level	CloudWatch Logs	IncidentTracking, Rescue Prioritization

Failure Handling — 2 ระดับ:

ระดับ 1: Lambda → EventBridge Publish ล้มเหลว

→ Outbox Pattern:

1. Lambda บันทึก Event ลง EventOutbox Table (DynamoDB)

```
{  
  
  "outbox_id": "OBX-uuid",  
  
  "event_type": "MissionStatusChanged",  
  
  "event_payload": "{...}",  
  
  "status": "PENDING",  
  
  "retry_count": 0,  
  
  "ttl": "<7 days>"  
}
```

2. [Demo 2+] outbox-processor Lambda

(CloudWatch Events trigger ทุก 1 นาที)

→ Scan PENDING → Retry publish → SENT / FAILED

3. [Demo 1] Records สะสมไว้ (ยังไม่มี processor)

ระดับ 2: EventBridge → Target Delivery ล้มเหลว

→ EventBridge built-in Retry (24 ชั่วโมง)

→ หากยังไม่สำเร็จ → สามารถตั้ง DLQ ที่ Rule level (Demo 2+)

สำคัญ: POST request ไม่ fail เพราะ EventBridge ล้มเหลว — ข้อมูลสถานะถูกบันทึกใน DynamoDB แล้ว

สรุป Dependency ทั้งหมด

#	Dependency	Type	Direction	Interaction	Criticality	Demo 1	Demo 2+
1	IncidentTracking	Service	ขาออก Sync + Async	HTTP GET + EventBridge Events	High	 Mock + CW Logs	 [TBD]
2	Dispatch Mgmt	Service	ขาเข้า + ขาออก Async + Sync GET	EventBridge Events + HTTP GET	Critical	 Seed Data + CW Logs	 [TBD]
3	Rescue Prioritization	Service	ขาออก Async	EventBridge Events	Medium	 CW Logs	 [TBD]
4	Amazon S3	Infrastructure	Internal	Presigned URL	Medium		
5	API GW + Authorizer	Infrastructure	Internal	Sync (REST)	Critical		
6	EventBridge	Infrastructure	Internal	Async (Events)	Critical	 (3 Rules → CW Logs)	 (+ Real Targets)

Krittamet (IncidentTracking Service)

เรื่องที่ 1: ผมจะเรียก API ของคุณ (Synchronous)

ตอนนี้ผม implement ให้ get-mission Lambda ของผมเรียก HTTP GET ไปหา IncidentTracking ของคุณ เพื่อดึงรายละเอียดเหตุการณ์มาแสดงให้ทีมกู้ภัยดู

API ที่ผมเรียก

```
GET {YOUR_SERVICE_URL}/incidents/{incident_id}
```

Response ที่ผมคาดหวัง (HTTP 200)

```
{
  "incident_id": "INC-001",
  "description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",
  "location": "13.7563,100.5018",
  "incident_type": "FLOOD"
}
```

คำถาม

1. API path ของคุณคือ GET /incidents/{incident_id} ถูกไหม? หรือใช้ path อื่น?
2. Response format ที่ผม list ไว้ข้างบน — field names ตรงกับที่คุณ implement ไหม? (incident_id, description, location, incident_type) ถ้าชื่อ field ต่างกัน บอกผมได้เลยจะได้แก้ไขให้ตรง
3. มี field อื่นที่คุณส่งกลับมาด้วยไหมที่ผมควรรู้? (เช่น impact_level, priority, status, created_at)
4. ถ้า incident_id ไม่มีในระบบ คุณ return HTTP status อะไร? (เช่น 404?) และ response body เป็นยังไง?
5. Service ของคุณ deploy แล้วหรือยัง? ถ้า deploy แล้ว — URL คืออะไร? (เช่น <https://xxxxx.execute-api.us-east-1.amazonaws.com/prod>) ผมจะได้เปลี่ยนจาก Mock URL ไปเป็น URL จริง (แก้ตัวแปร Terraform แล้ว apply ใหม่ ก็ใช้ได้เลย)
6. ถ้ายัง deploy ไม่เสร็จ — คาดว่าจะพร้อมเมื่อไหร่? (ผม Mock ไว้แล้วรอเปลี่ยน URL ได้เลย)

หมายเหตุ: ผมออกแบบ Degraded Mode ไว้แล้ว — ถ้าเรียก API ของคุณไม่สำเร็จ (timeout / error) ระบบของผมยังทำงานได้ปกติ แค่แสดงข้อมูลไม่ครบ (data_source: "partial") ทีมกู้ภัยยังอัปเดตสถานะได้

เรื่องที่ 2: ผมจะส่ง Events ให้คุณ (Asynchronous)

เมื่อทีมกู้ภัยอัปเดตสถานะภารกิจ ผมจะ publish events ไปยัง Amazon EventBridge (custom event bus: mission-progress-events)

Events ที่คุณน่าจะสนใจ:

Event 1: MissionStatusChanged

```
{
  "source": "mission-progress-service",
  "detail-type": "MissionStatusChanged",
  "detail": {
    "mission_id": "MSN-001",
    "incident_id": "INC-001",
    "rescue_team_id": "TEAM-ALPHA",
    "old_status": "EN_ROUTE",
    "new_status": "ON_SITE",
    "note": "ถึงจุดเกิดเหตุแล้ว",
    "updated_at": "2025-06-14T09:32:15Z",
    "performed_by": "TEAM-ALPHA"
  }
}
```

ประโยชน์สำหรับคุณ: อัปเดตสถานะรวมของ Incident (เช่น เปลี่ยนเป็น "In Progress", "Resolved")

Event 2: ImpactLevelUpdated

```
{
  "source": "mission-progress-service",
  "detail-type": "ImpactLevelUpdated",
  "detail": {
    "mission_id": "MSN-001",
    "incident_id": "INC-001",
    "rescue_team_id": "TEAM-ALPHA",
    "new_impact_level": "HIGH",
    "note": "น้ำเพิ่มระดับเร็วกว่าที่ประเมิน",
    "updated_at": "2025-06-14T09:35:00Z"
  }
}
```

ประโยชน์สำหรับคุณ: ทีมกู้ภัยประเมินความรุนแรงจากหน้างานแล้วส่งค่าใหม่มา → คุณ (เป็น Source of Truth) อัปเดต Impact Level ได้

คำถาม

2.1 คุณสนใจรับ Event MissionStatusChanged ไหม? ถ้าสนใจ — คุณจะตั้ง EventBridge Rule ที่ bus mission-progress-events เพื่อ route event มาหา Service ของคุณเองได้เลย

2.2 คุณสนใจรับ Event ImpactLevelUpdated ไหม? (เพื่ออัปเดต Impact Level ใน Incident Master Data)

2.3 Payload field names / format OK ไหม? อยากได้ field เพิ่มบอกได้เลย

2.4 ถ้าสนใจ — คุณสามารถตั้ง EventBridge Rule ที่ bus mission-progress-events เพื่อ route event มาหา Service ของคุณได้เลย (เช่น target เป็น Lambda หรือ SQS ของคุณ) คุณถนัดตั้ง Rule เอง หรืออยากให้ผมช่วย?

Noppakron (Dispatch Management Service)

เรื่องที่ 1: ผมต้องการรับ Event จากคุณ (Inbound — คุณส่งมาให้ผม)

เมื่อ Dispatcher มอบหมายภารกิจ ให้ทีมกู้ภัย ผมต้องการรู้ว่ามี Mission ใหม่เกิดขึ้น เพื่อสร้าง Mission Record ในระบบของผม

Event ที่ผมอยากได้ (ตั้งชื่อว่า MissionAssignedEvent):

```
{
  "source": "dispatch-management-service",
  "detail-type": "MissionAssignedEvent",
  "detail": {
    "mission_id": "MSN-001",
    "rescue_unit_id": "TEAM-ALPHA",
    "incident_id": "INC-001",
    "incident_type": "FLOOD",
    "incident_description": "น้ำท่วมหนัก บ้าน 2 ชั้น ชั้นล่างจม",
    "incident_location": "13.7563,100.5018",
    "impact_level": "MODERATE",
    "priority": "MEDIUM",
    "assigned_at": "2025-06-14T08:45:00Z"
  }
}
```

ผมจะเอาข้อมูลนี้ไปทำอะไร:

- สร้าง Mission Record ใน MissionAssignment table (status = DISPATCHED)
- สร้าง Timeline Entry แรก ("Mission assigned to TEAM-ALPHA")
- เก็บ incident data ไว้แสดงให้ทีมกู้ภัยดู (ไม่ต้องไปดึงจาก IncidentTracking อีกรอบ)

คำถาม

1.1 เมื่อ Dispatcher มอบหมายงาน — คุณ publish event ออกมาไหม? ถ้า publish — ชื่อ event คืออะไร?

1.2 คุณ publish ผ่านช่องทางอะไร? (EventBridge / SNS / SQS / อื่นๆ?) ถ้า EventBridge — ใช้ bus ชื่ออะไร?

1.3 Payload ที่คุณส่งมา มี field อะไรบ้าง? ผม list ไว้ข้างบนว่าผมอยากได้อะไร คุณส่งมาครบไหม? field ไหนไม่มีบ้าง?

1.4 ชื่อ field ตรงกับที่ผม list ไหม? เช่น คุณใช้ team_id หรือ rescue_unit_id? ใช้ incident_id หรือชื่ออื่น?

1.5 mission_id ใครเป็นคน generate? คุณ generate มาให้ผมเลย? หรือผมต้อง generate เอง?

1.6 ถ้าคุณยังไม่ได้ publish event → ตอนนี้ผม Mock ด้วย Seed Data (script insert ข้อมูลตัวอย่างลง DynamoDB ตรง) → พอ Demo 2+ ค่อยเปลี่ยนเป็นรับ Event จริง → คุณ OK ไหม?

1.7 คาดว่า Service ของคุณจะ deploy + publish event ได้เมื่อไหร่?

เรื่องที่ 2: ผมจะส่ง Event ให้คุณ (Outbound — ผมส่งไปหาคุณ)

เมื่อทีมกู้ภัยอัปเดตสถานะเป็น RESOLVED (ภารกิจเสร็จสิ้น) ผมจะ publish event ให้คุณเพื่อ ปลดล็อกทีมกู้ภัย ให้กลับมาเป็น Available

Event: MissionStatusChanged (เฉพาะ RESOLVED)

```
{
```

```
  "source": "mission-progress-service",
```

```
  "detail-type": "MissionStatusChanged",
```

```
  "detail": {
```

```
    "mission_id": "MSN-001",
```

```
    "incident_id": "INC-001",
```

```
    "rescue_team_id": "TEAM-ALPHA",
```

```
    "old_status": "ON_SITE",
```

```
"new_status": "RESOLVED",
```

```
"note": "ภารกิจเสร็จสิ้น อพยพผู้ประสบภัยครบแล้ว",
```

```
"updated_at": "2025-06-14T12:00:00Z",
```

```
"performed_by": "TEAM-ALPHA"
```

```
}
```

```
}
```

คำถาม

2.1 คุณสนใจรับ Event นี้ไหม? (เพื่อปลดล็อกทีมกู้ภัยจาก BUSY → AVAILABLE)

2.2 คุณต้องการรับเฉพาะ RESOLVED หรือสนใจสถานะอื่นด้วย? (เช่น NEED_BACKUP เพื่อรู้ว่าทีมต้องการกำลังเสริม?)

2.3 Payload field names / format OK ไหม? อยากได้ field เพิ่มบอกได้เลย

2.4 ถ้าสนใจ — คุณตั้ง EventBridge Rule ที่ bus mission-progress-events ได้เลย โดย filter "detail.new_status": ["RESOLVED"] → คุณถนัดตั้ง Rule เอง หรืออยากให้ผมช่วย?

เรื่องที่ 3: คุณต้องการเรียก API ของผมไหม? (Synchronous)

ผมมี API:

GET /incidents/{incident_id}

ส่งกลับ: Mission state + Timeline ทั้งหมด + รูปภาพหลักฐาน (Demo 2+)

Use case สำหรับคุณ: ถ้า Dispatcher/ผู้สั่งการ ต้องการดู Timeline ละเอียด ของภารกิจ (Log ทุกรายการ, เวลา, ผู้กระทำ, รูปภาพ) — สามารถเรียก API นี้ได้

Response ตัวอย่าง

```
{  
  
  "incident_id": "INC-001",  
  
  "mission_id": "MSN-001",  
  
  "rescue_team_id": "TEAM-ALPHA",  
  
  "current_status": "ON_SITE",  
  
  "latest_impact_level": 3,  
  
  "started_at": "2024-12-01T08:00:00Z",  
  
  "last_updated_at": "2025-06-14T09:32:15Z",  
  
  "timeline": [  
  
    {  
  
      "timestamp": "2024-12-01T08:00:00Z",  
  
      "action_type": "STATUS_CHANGE",  
  
      "description": "Mission dispatched to TEAM-ALPHA",  

```



```
"performed_by": "SYSTEM"
```

```
},
```

```
{
```

```
"timestamp": "2025-06-14T09:32:15Z",
```

```
"action_type": "STATUS_CHANGE",
```

```
"description": "Arrived at incident location",
```

```
"performed_by": "TEAM-ALPHA"
```

```
}
```

```
],
```

```
"data_source": "partial"
```

```
}
```

คำถาม

3.1 Service ของคุณ (หรือ Dashboard) ต้องการเรียก API ของผมเพื่อดู Timeline ละเอียดไหม?

3.2 ถ้าต้องการ — คุณจะเรียกจาก Backend ของคุณ (Lambda→Lambda) หรือจาก Frontend ของคุณ?

3.3 ถ้าเรียกจาก Backend — ผมต้องให้ API Key กับคุณ (เพราะ API ผมต้อง auth) → คุณ OK ไหม?

Nattasak (Rescue Prioritization Service)

เรื่องที่ 1: ผมจะส่ง Events ให้คุณ (Asynchronous)

เมื่อทีมกู้ภัยอัปเดตข้อมูลจากหน้างาน ผมจะ publish events ไปยัง Amazon EventBridge (custom event bus: mission-progress-events) ที่น่าจะเป็นประโยชน์กับ Rescue Prioritization ของคุณ:

Event 1: MissionBackupRequested

Trigger: ทีมกู้ภัยยกสถานะ NEED_BACKUP (ต้องการกำลังเสริม)

```
{  
  
  "source": "mission-progress-service",  
  
  "detail-type": "MissionBackupRequested",  
  
  "detail": {  
  
    "mission_id": "MSN-001",  
  
    "incident_id": "INC-001",  
  
    "rescue_team_id": "TEAM-ALPHA",  
  
    "old_status": "ON_SITE",  
  
    "new_status": "NEED_BACKUP",  
  
    "note": "น้ำเพิ่มระดับเร็ว ต้องการเรือเพิ่ม 2 ลำ",  
  
    "updated_at": "2025-06-14T10:15:00Z",  
  
    "performed_by": "TEAM-ALPHA"  
  
  }  
}
```

ประโยชน์สำหรับคุณ: รู้ว่าเคสนี้ต้องการกำลังเสริม → อาจนำไปปรับ Priority Score

Event 2: ImpactLevelUpdated

Trigger: ทีมกู้ภัย ปรับระดับความรุนแรง จากหน้างาน (เพราะประเมินจากข้อมูลเดิมแล้วไม่ตรงกับความเป็นจริง)

```
{  
  
  "source": "mission-progress-service",  
  
  "detail-type": "ImpactLevelUpdated",  
  
  "detail": {  
  
    "mission_id": "MSN-001",  
  
    "incident_id": "INC-001",  
  
    "rescue_team_id": "TEAM-ALPHA",  
  
    "new_impact_level": "HIGH",  
  
    "note": "น้ำเพิ่มระดับเร็วกว่าที่ประเมิน มีผู้สูงอายุติดอยู่ชั้น 2",  
  
    "updated_at": "2025-06-14T09:35:00Z",  
  
    "performed_by": "TEAM-ALPHA"  
  }  
}
```

ประโยชน์สำหรับคุณ: ได้ค่า Impact Level ล่าสุดจากหน้างาน (ค่าจริง ไม่ใช่ค่าประเมินเริ่มต้น) → นำไปคำนวณ Priority Score ใหม่

คำถาม

- 1.1 คุณสนใจรับ Event MissionBackupRequested ไหม? (เพื่อรู้ว่าเคสไหนต้องการกำลังเสริม)
- 1.2 คุณสนใจรับ Event ImpactLevelUpdated ไหม? (เพื่อได้ค่า Impact Level ล่าสุดจากหน้างาน)
- 1.3 นอกจาก 2 Events นี้ — คุณสนใจ MissionStatusChanged ด้วยไหม? (เช่น อยากรู้เมื่อภารกิจ RESOLVED เพื่อเอาเคสออกจากคิว?)
- 1.4 Payload field names / format OK ไหม? อยากได้ field เพิ่มบอกได้เลย
- 1.5 new_impact_level ผมส่งเป็น String ("HIGH", "MEDIUM", "LOW") — คุณรับ format นี้ได้ไหม? หรือต้องการเป็นตัวเลข?

1.6 ถ้าสนใจ — คุณตั้ง EventBridge Rule ที่ bus mission-progress-events ได้เลย → คุณถนัดตั้ง Rule เอง หรืออยากให้ผมช่วย?

เรื่องที่ 2: ผมต้องการข้อมูลจากคุณไหม?

ตอนนี้ผมคิดว่า MissionProgress ไม่ต้องเรียก API ของ Rescue Prioritization (ไม่มี use case ที่ต้องดู Priority Score ในหน้า Web App ของทีมกู้ภัย)

แต่ถ้าอนาคต ทีมกู้ภัยอยากเห็น Priority Score ของเคสที่ตัวเองกำลังทำ → ผมอาจต้องเรียก API ของคุณ

คำถาม

2.1 คุณมี API สำหรับดึง Priority Score ของ Incident ไหม? (เช่น GET /priorities/{incident_id})

2.2 ถ้ามี — Response format เป็นยังไง? (เผื่ออนาคตผมต้องเรียก)

2.3 หรือคุณ publish event เมื่อ Priority Score เปลี่ยนแปลง? (เช่น PriorityScoreUpdated) → ผมสามารถ subscribe ได้

Github Public Repo

<https://github.com/Ratthatummanoon/CS366-MissionProgress-Service>